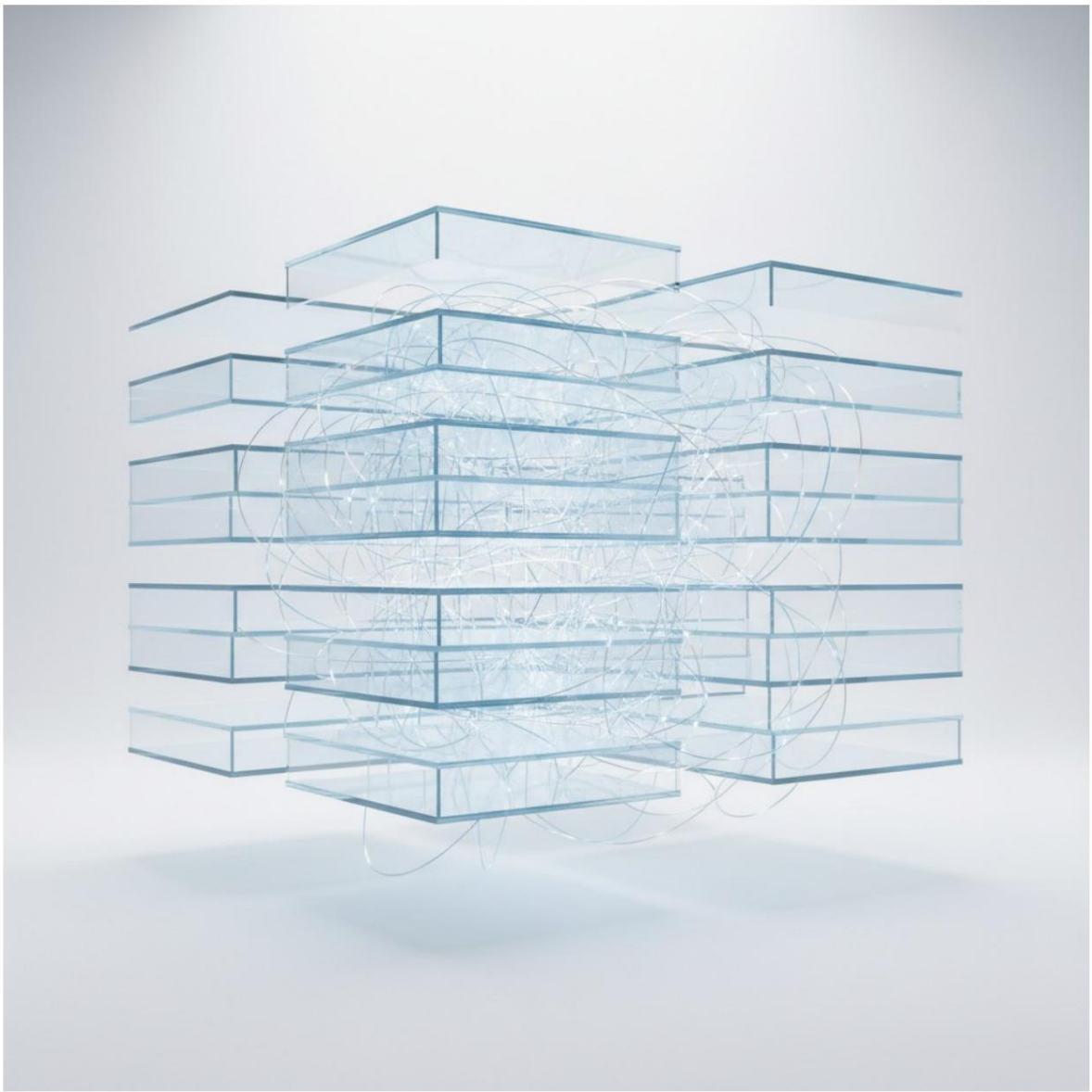
Machine Translated by Google

Relacyjne bazy danych



2026 – Adrián Esneyder Salcedo

Machine Translated by Google

Program kursu: Bazy danych relacyjne

Moduł 1: Podstawy baz danych

- Podstawowe pojęcia: dane, informacje i wiedza
- Czym jest baza danych i system zarządzania bazą danych (DBMS)?
- Historyczna ewolucja baz danych
- Bazy danych relacyjne i nierelacyjne
- Zalety i przypadki użycia relacyjnych baz danych

Moduł 2: Model relacyjny

- Składniki modelu relacyjnego: tabele, wiersze i kolumny
- Podstawowe koncepcje: krotki, atrybuty, domeny
- Klucze podstawowe i klucze obce
- Typy relacji: jeden do jednego (1:1), jeden do wielu (1:N), wiele do wielu (N:M)
- Integralność referencyjna i ograniczenia (UNIQUE, NOT NULL, CHECK)

Moduł 3: Koncepcyjne projektowanie baz danych

- Analiza wymagań systemowych
- Model relacji encji (ER): encje, atrybuty i relacje
- Diagramy ER: oznaczenia i symbolika
- Kardynalność i uczestnictwo w relacjach
- Słabe i silne podmioty

Moduł 4: Projektowanie logiczne i standaryzacja

- Transformacja z modelu ER do modelu relacyjnego
- Zależności funkcjonalne
- Postacie normalne: 1NF, 2NF, 3NF i BCNF (Boyce-Codd)
- Proces normalizacji i denormalizacji
- Projektowanie efektywnych schematów.

Moduł 5: SQL – Język definicji danych (DDL)

- Wprowadzenie do standardowego SQL
- Tworzenie baz danych: UTWÓRZ BAZĘ DANYCH

Machine Translated by Google

- Tworzenie i modyfikowanie tabel: CREATE TABLE, ALTER TABLE, DROP TABLE
- Typy danych: VARCHAR, INT, DATE, DECIMAL, BOOLEAN itp.
- Definicja ograniczeń i indeksu

Moduł 6: SQL – Język manipulacji danymi (DML)

- Wprowadzanie danych: INSERT INTO
- Aktualizacja rekordu: AKTUALIZACJA
- Usuwanie danych: USUŃ
- Transakcje: BEGIN, COMMIT, ROLLBACK
- Właściwości ACID (atomowość, konsystencja, izolacyjność, trwałość)

Moduł 7: Podstawowe zapytania SQL

- Polecenie SELECT: składnia i struktura
- Filtrowanie danych: klauzula WHERE
- Sortowanie: ORDER BY (rosnąco, malejąco)
- Operatory porównania i logiczne: AND, OR, NOT, IN, BETWEEN, LIKE
- Funkcje agregacji: COUNT(), SUM(), AVG(), MAX(), MIN()
- Grupowanie: GROUP BY i HAVING

Moduł 8: Zaawansowane zapytania SQL

- Połączenia (unie tabel): INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL POŁĄCZENIE ZEWNĘTRZNE
- Podzapytania (subquery): w SELECT, WHERE i FROM
- Operatorzy zbiorów: UNION, INTERSECT, EXCEPT
- Aliasy tabel i kolumn (AS)
- Zapytania zagnieżdżone i skorelowane

Moduł 9: Funkcje składowane i procedury

- Widoki: tworzenie i zalety
- Funkcje zdefiniowane przez użytkownika (UDF)
- Procedury składowane: parametry wejścia/wyjścia
- Wyzwalacze: zdarzenia i automatyzacja
- Zmienne i struktury sterujące (IF, WHILE, CASE)

Machine Translated by Google

Moduł 10: Optymalizacja i wydajność • Indeksy: typy i strategie indeksowania

- Analiza planów wykonania (WYJAŚNIJ)
- Optymalizacja zapytań SQL
- Partycjonowanie tabeli
- Dobre praktyki projektowe dla wydajności

Moduł 11: Bezpieczeństwo i administracja

- Użytkownicy i uprawnienia: UDZIEL, COFNIJ
- Role i uprawnienia
- Tworzenie kopii zapasowych i odzyskiwanie danych (kopia zapasowa/przywracanie)
- Strategie migracji baz danych
- Audyt i rejestrowanie działań

Moduł 12: Ostateczny projekt praktyczny

- Analiza rzeczywistego przypadku biznesowego
- Kompleksowy projekt: od modelu ER do wdrożenia
- Utworzenie bazy danych z danymi testowymi
- Implementacja złożonych zapytań
- Dokumentacja techniczna projektu

Machine Translated by Google

1. Składniki modelu relacyjnego: tabele, wiersze i kolumny

Model relacyjny opiera się na idei reprezentacji danych jako relacji matematycznych, ale w praktyce przekłada się to na proste i wizualne struktury: tabele, wiersze i kolumny. Komponenty te stanowią podstawę każdej relacyjnej bazy danych i umożliwiają logiczną i przystępną organizację informacji. W MySQL
Tabele są przechowywane głównie w silniku InnoDB (domyślnym od wersji MySQL 5.5), który obsługuje transakcje ACID i klucze obce.

Tabele (relacje)

Tabela to podstawowa jednostka pamięci w modelu relacyjnym. Reprezentuje ona encję lub zbiór powiązanych obiektów, takich jak „Pracownicy", „Produkty" lub „Sprzedaż". Każda tabela to zbiór danych uporządkowanych w formacie dwuwymiarowym.
Podobne do arkusza kalkulacyjnego, ale z rygorystycznymi zasadami gwarantującymi spójność.

Kluczowe właściwości tabeli w MySQL

Jest to nieuporządkowany zbiór wierszy (nie ma określonej kolejności, choć można go sortować za pomocą zapytań, takich jak ORDER BY).

Wszystkie wiersze w tabeli muszą mieć tę samą strukturę (tę samą liczbę i ten sam typ kolumn).

Tabele są definiowane w bazie danych pod unikalną nazwą i mogą korzystać z mechanizmów takich jak InnoDB w celu zapewnienia integralności referencyjnej.

Wiersze (rekordy lub krotki)

Każdy wiersz reprezentuje unikatową instancję encji, którą opisuje tabela. Na przykład w tabeli „Pracownicy" wiersz może zawierać dane konkretnego pracownika: identyfikator, imię, nazwisko, stanowisko itp.

Funkcje w MySQL

Wiersze są atomowe: każdy z nich jest niepodzielny i przedstawia fakt kompletny.

W dobrze zaprojektowanej tabeli nie są dozwolone żadne duplikaty (dzięki kluczom podstawowym).

Liczba wierszy może zmieniać się dynamicznie (można je wstawiać, aktualizować lub usuwać za pomocą poleceń takich jak INSERT, UPDATE i DELETE).

Machine Translated by Google

Kolumny (atrybuty lub pola)

Każda kolumna definiuje właściwość lub cechę jednostki. Na przykład w tabeli „Pracownicy” kolumny mogą brzmieć: „Identyfikator pracownika”, „Imię”, „Nazwisko”, „Wynagrodzenie”.

Funkcje w MySQL

Każda kolumna ma unikalną nazwę w tabeli.

Jest on powiązany z typem danych (np. INT, VARCHAR, DECIMAL, DATE), który definiuje, jakie wartości może on przechowywać.

Kolumny definiują „schemat” tabeli, który musi zostać ustalony, choć można go zmienić za pomocą polecenia ALTER TABLE.

Przykład praktyczny Wyobraźmy sobie bazę danych sklepu internetowego. Tabela „Produkty” może wyglądać następująco (reprezentacja tekstowa):

ID_produktu	Nazwa	Cena	Magazyn
	Laptop XYZ	1200	50
1	Mysz ABC	25	200
2 3	Klawiatura DEF	50	100

- Tutaj tabela nosi nazwę „Produkty”.
- Każdy wiersz przedstawia konkretny produkt.
- Kolumny zawierają właściwości (Product_ID, Nazwa itp.).

W MySQL utworzenie tej tabeli wyglądałoby następująco:

```
1 UTWÓRZ TABELĘ Produkty (
2 Product_ID INT AUTO_INCREMENT PRIMARY KEY ,
3 Nazwa VARCHAR(100),
4 DECIMAL Cena (10, 2),
5 akcji INT
6 ) SILNIK=InnoDB;
```

Ten komponent zapewnia łatwość wyszukiwania i manipulowania danymi za pomocą operacji takich jak SELECT, INSERT, UPDATE i DELETE. W MySQL należy w tym celu użyć ENGINE=InnoDB . pełne wsparcie dla transakcji i kluczy obcych.

Machine Translated by Google

2. Podstawowe koncepcje: krotki, atrybuty i domeny

Aby zrozumieć model relacyjny z bardziej formalnej perspektywy, wprowadzamy trzy kluczowe pojęcia wywodzące się z teorii mnogości i relacji matematycznych: krotki, atrybuty i domeny. Terminy te stanowią podstawę teoretyczną, która wspiera praktyczną implementację w systemach RDBMS, takich jak MySQL.

Krotki: Krotka to formalny odpowiednik wiersza w tabeli. Jest to uporządkowany zbiór wartości, gdzie każda wartość odpowiada określonemu atrybutowi. W matematyce relacja to podzbiór iloczynu kartezjańskiego dziedzin, a każda
Krotka jest elementem tej relacji.

Właściwości w MySQL:

Krotki są unikalne w relacji (nie ma duplikatów).

Teoretycznie są one niezmienne, jednak w praktyce można je aktualizować za pomocą polecenia UPDATE.

Przykład: W tabeli „Produkty" powyżej krotka dla pierwszego produktu to (1, 'Laptop XYZ', 1200, 50).

Atrybuty: Atrybut jest formalnym odpowiednikiem kolumny. Reprezentuje właściwość encji i jest powiązany z określoną domeną.

Właściwości w MySQL:

Atrybuty definiują stopień (liczbę kolumn) relacji.

Każdy atrybut ma rolę semantyczną (np. „ID_Product" jednoznacznie identyfikuje).

W MySQL są one definiowane za pomocą typów danych, które przybliżają domeny, takich jak TINYINT dla małe wartości lub ENUM dla domen ograniczonych.

Domeny: Domena to zbiór możliwych wartości atrybutu. Definiuje ona ograniczenia dotyczące tego, jakie dane mogą być przechowywane w kolumnie, zapewniając spójność.

Właściwości w MySQL:

Może być skończony (np. ENUM('Mężczyzna', 'Kobieta') w przypadku słowa „Płeć") lub nieskończony (np. wszystkie liczby całkowite w przypadku słowa „Wiek").

Zawiera dodatkowe typy danych i ograniczenia (np. Wiek > 0 przy użyciu wyzwalaczy, ponieważ CHECK ma ograniczenia w starszych wersjach).

MySQL aproksymuje domeny za pomocą typów, takich jak INT (liczby całkowite), VARCHAR (ciągi znaków), BOOLEAN itd., oraz typów, takich jak ENUM lub SET, w przypadku predefiniowanych list.

Przykład praktyczny: dla tabeli „Klienci":

Atrybuty: Client_ID (domena: liczby całkowite dodatnie), Nazwa (domena: ciągi znaków do 50 znaków), Wiek (domena: liczby całkowite od 18 do 100).

Krotka: (101, „Juan Pérez", 35).

Machine Translated by Google

Koncepcje te zapewniają, że model relacyjny jest matematycznie rygorystyczny, co pozwala na dowodzenie poprawności i optymalizację zapytań MySQL.

3. Klucze podstawowe i klucze obce

Klucze to mechanizmy zapewniające unikalność, integralność i relacje między danymi. Są one niezbędne do zapobiegania redundancji i błędom. W MySQL z InnoDB klucze obce wymagają ustawienia ENGINE=InnoDB.

Klucze podstawowe : Klucz podstawowy to atrybut (lub zestaw atrybutów), który jednoznacznie identyfikuje każdą krotkę w tabeli. Jest to „główny identyfikator".

Funkcje MySQL:

Musi być unikatowy i nie może być nullem (NOT NULL jest domyślny).

Zazwyczaj jest to automatycznie zwiększający się identyfikator numeryczny (AUTO_INCREMENT).

Może być prosty (jeden atrybut) lub złożony (kilka atrybutów).

Korzyści: Przyspiesza wyszukiwanie dzięki automatycznemu indeksowaniu.

Przykład w MySQL:

```
1 UTWÓRZ TABELĘ Pracownicy (
2 Employee_ID INT AUTO_INCREMENT PRIMARY KEY ,
3 Nazwa VARCHAR(50),
4 Nazwisko VARCHAR(50)
5 ) SILNIK=InnoDB;
```

Klucze obce (KLUCZ OBCY): Klucz obcy to atrybut w jednej tabeli, który odwołuje się do klucza podstawowego innej tabeli, ustanawiając między nimi relację.

Funkcje MySQL:

Sprawdza, czy wartości w kolumnie obcej istnieją w tabeli, do której się odwołuje (wymaga InnoDB).

Może być nullem (jeśli relacja jest opcjonalna).

Obsługuje akcje kaskadowe (np. ON DELETE CASCADE).

W starszych wersjach sprawdź, czy tabele korzystają z tego samego silnika.

Machine Translated by Google

Przykład w MySQL:

W tabeli „Zamówienia” odwołującej się do „Klientów”:

```
UTWÓRZ TABELĘ Zamówienia (
Order_ID INT AUTO_INCREMENT PRIMARY KEY ,
    ID_Klient INT,
Data DATA,
ODNIESIENIA DO KLUCZA OBCEGO (ID_KLIENTA) Klienci (ID_KLIENTA) PRZY USUWANIU KASKADOWYM
) SILNIK=InnoDB;
```

Klucze te stanowią podstawę normalizacji baz danych i unikania anomalii w MySQL.

4. Typy relacji: Jeden do jednego (1:1), Jeden do wielu (1:N), Wiele do wielu (N:M)

Relacje określają sposób, w jaki jednostki są połączone poprzez klucze obce.

Są one klasyfikowane według kardynalności (liczba elementów jednej encji jest powiązana z liczbą elementów innej encji). W MySQL relacje są implementowane za pomocą kluczy obcych ; w InnoDB są one implementowane za pomocą kluczy obcych .

Jeden do jednego (1:1): Każde wystąpienie jednego bytu jest powiązane z dokładnie jednym wystąpieniem innego. Przykład: Pracownik ma przypisane tylko jedno biurko.

Implementacja MySQL: Klucz obcy umieszczany jest w jednej z tabel lub w celu wymuszenia stosunku 1:1 używana jest współdzielona tabela z atrybutem UNIQUE.

Przykład: Tabela „Pracownicy” i „Dane osobowe” (ID_Employee jako PK w obu z KLUCZEM OBCYM).

Jeden do wielu (1:N): Jeden egzemplarz encji jest powiązany z wieloma egzemplarzami innego, ale nie odwrotnie. Przykład: Klient może mieć wiele zamówień.

Implementacja MySQL: Klucz obcy w tabeli „many” (np. ID_Client w tabeli „Orders”).

Przykład w MySQL: Jak w poprzednim punkcie.

Wiele do wielu (N:M): Wiele instancji jednego bytu jest powiązanych z wieloma instancjami innego. Przykład: Studenci i kursy (student uczęszcza na kilka kursów, kurs ma wielu studentów).

Implementacja MySQL: Wymaga tabeli pośredniej (tabeli połączeniowej) zawierającej klucze obce z obu tabel.

Machine Translated by Google

Przykład w MySQL:

```
UTWÓRZ TABELĘ Studenci_Kursy (
    Identyfikator_studenta INT,
ID_Kurs INT,
KLUCZ PODSTAWOWY (ID_Estudiante, ID_Curso),
KLUCZ OBCY (ID_Estudiante) REFERENCJE Studenci (ID_Estudiante),
KLUCZ OBCY (ID_KURSU) REFERENCJE Kursy (ID_KURSU)
) SILNIK=InnoDB;
```

Zrozumienie tych zależności jest kluczowe dla projektowania znormalizowanych schematów w MySQL.

5. Integralność referencyjna i ograniczenia (UNIQUE, NOT NULL, CHECK)

Integralność danych gwarantuje ich dokładność, spójność i poprawność. Model relacyjny narzuca reguły, aby zapobiec naruszeniom. W MySQL integralność referencyjna opiera się na InnoDB.

Integralność referencyjna: zapewnia, że klucze obce wskazują na istniejące krotki w tabeli, do której się odwołują. Zapobiega powstawaniu „sierot" (np. zamówienia bez klienta).

Wymuszanie w MySQL: Używanie klucza obcego (FOEIGN KEY) z opcjami takimi jak RESTRICT, CASCADE, SET NULL. Wymaga InnoDB; w MyISAM jest ignorowane.

Przykład: próba wstawienia zamówienia z nieistniejącym ID_Cliente zakończy się niepowodzeniem, jeśli ta opcja jest włączona.

Ograniczenia:

UNIQUE: Zapewnia unikalność wartości w kolumnie (lub zestawie). Podobny do PRIMARY KEY, ale dopuszcza wartości NULL.

Przykład: E-mail VARCHAR(100) UNIQUE.

NOT NULL: Zabrania wartości null w kolumnie.

Przykład: Nazwa VARCHAR(50) NOT NULL.

SPRAWDZAĆ:

Definiuje niestandardowy warunek, który muszą spełniać wartości. Ostrzeżenie w MySQL: Dotyczy to w pełni wersji 8.0.16 (kwiecień 2019 r.). We wcześniejszych wersjach (np. 5.7 lub 8.0.15-) składnia jest akceptowana, ale całkowicie ignorowana.
Sprawdź za pomocą SELECT VERSION();.

Przykład (dla MySQL 8.0.16+): Wiek INT CHECK (Wiek >= 18).

Alternatywa dla starszych wersji lub w celu zapewnienia zgodności: Użyj wyzwalaczy, aby symulować CHECK.SQLDELIMITER //

Machine Translated by Google

```
UTWÓRZ WYZWALACZ trg_age_before_insert
PRZED WŁOŻENIEM DO PRACOWNIKÓW
DLA KAŻDEGO RZĘDU
ZACZYNAĆ
    JEŚLI NOWY.Wiek < 18 WTEDY
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Wiek musi być większy lub równy 18';

    KONIEC JEŻELI;
KONIEC //
DELIMITER ; Tworzy podobny wyzwalacz dla BEFORE UPDATE.
```

W MySQL definiuje się je podczas tworzenia tabeli:

```
SQLCREATE TABLE Użytkownicy (
    ID INT AUTO_INCREMENT KLUCZA PODSTAWOWEGO,
    Nazwa użytkownika VARCHAR(50) UNIQUE NOT NULL,
    Wiek INT CHECK (wiek      18) — Działa tylko w MySQL 8.0.16+
) SILNIK=InnoDB;
```

Te ograniczenia są automatycznie sprawdzane przez MySQL (jeśli ma to zastosowanie), co sprzyja wiarygodności danych. Aby zwiększyć integralność danych w starszych wersjach, należy nadać priorytet wyzwalaczom lub walidacji aplikacji.

Wnioski z modułu

W tym module dostosowanym do MySQL omówiliśmy podstawy modelu relacyjnego, od jego podstawowych komponentów po reguły utrzymujące jego integralność,
Z naciskiem na kompatybilność i ograniczenia MySQL. Dzięki temu fundamentowi będziesz gotowy do projektowania relacyjnych baz danych w MySQL w następnym module. Wykonaj ćwiczenia praktyczne: twórz tabele z relacjami i ograniczeniami w MySQL. W razie pytań przejrzyj przykłady lub poeksperymentuj z zapytaniami SQL.

Gdy będziesz gotowy, przejdziemy do następnego modułu!

Różnice między SQL, MySQL i PostgreSQL

Aby nadać kontekst, najpierw wyjaśnijmy pojęcia: SQL nie jest systemem baz danych, lecz standardowym językiem (Structured Query Language) do zarządzania relacyjnymi bazami danych. Jest to standard ANSI/ISO, który definiuje polecenia takie jak SELECT, INSERT itp. MySQL i PostgreSQL to systemy zarządzania relacyjnymi bazami danych (RDBMS), które implementują SQL, ale z wariantami, rozszerzeniami i różnicami w funkcjach, wydajności i zgodności ze standardem. Następnie,

Machine Translated by Google

Szczegółowe i profesjonalne porównanie oparte na stanie bieżącym (luty 2026 r.) i uwzględniające stabilne wersje, takie jak MySQL 8.4, PostgreSQL 18 i SQL Standard (SQL:2023).

Aby dokonać jasnego porównania, posłużę się tabelą:

Machine Translated by Google

Machine Translated by Google

Aspekty: SQL (Standard), MySQL (Oracle), PostgreSQL (Community). Definicja i cel:
Standardowy język do tworzenia zapytań i manipulowania danymi relacyjnymi (nie
oprogramowanie). Definiuje podstawową składnię, taką jak CREATE TABLE, SELECT itp. System
RDBMS typu open source (z edycjami Enterprise) implementujący SQL. Popularny w internecie
(np. stos LAMP). Skoncentrowany na prostocie i szybkości odczytu. Zaawansowany system RDBMS typu open source.
implementujący SQL. Skoncentrowany na standardach, rozszerzalności i funkcjach korporacyjnych,
takich jak GIS, JSON i wyszukiwanie pełnotekstowe. Zgodność ze standardem SQL: Jest standardem
samym w sobie (ANSI/ISO SQL:2023 obejmuje JSON, funkcje okienkowe itp.). Jest częściowo
zgodny; posiada zastrzeżone rozszerzenia (np. GROUP BY bez pełnych agregacji do wersji 8.0).
Ignoruje niektóre ścisłe reguły, aby być bardziej „przyjazny dla użytkownika”. Jest wysoce zgodny;
ściśle przestrzega standardu SQL i rozszerza go o funkcje, takie jak niestandardowe
typy i operatory. Lepszy pod względem zgodności. Silniki pamięci masowej: Nie dotyczy
(to język). Obsługuje wiele silników (domyślny InnoDB dla transakcji; MyISAM dla nietransakcyjnych).
Elastyczny, ale mniej spójny. Wykorzystuje jeden zintegrowany silnik z rozszerzeniami (np.
PostGIS dla danych geoprzestrzennych). Bardziej ujednolicony i rozszerzalny poprzez

Machine Translated by Google

Wtyczki. Transakcje i integralność. Definiuje ACID, ale implementacja może być różna. Obsługuje ACID. W InnoDB. CHECK tylko od wersji 8.0.16; FOREIGN KEY w InnoDB. Mniej rygorystyczny w trybie nierygorystycznym (np. dyskretnie dopuszcza nieprawidłowe dane). Obsługuje pełne ACID.
CHECK, FOREIGN KEY i zaawansowane wyzwalacze były zawsze obsługiwane. Bardzo rygorystyczne; domyślnie odrzuca nieprawidłowe dane. Typy danych i funkcje: Podstawowe: INT, VARCHAR, DATE itp. Rozszerzenia takie jak JSON w SQL: 2016. Dobre typy podstawowe + rozszerzenia takie jak JSON (od 5.7). Funkcje takie jak GROUP_CONCAT. Nie obsługuje tablic natywnych ani typów złożonych bez JSON. Typy zaawansowane: tablice, hstore (klucz-wartość), JSONB (efektywny binarny), XML, UUID. Funkcje okienkowe, rekurencyjne CTE i operatory niestandardowe. Wydajność i skalowalność: Nie dotyczy. Doskonałe do dużych odczytów i replikacji (np. MySQL Cluster). Mniej wydajne w przypadku złożonych złączeń lub analiz. Dobre do złożonych zapytań, OLAP i wysokiej współbieżności. Obsługuje deklaratywne partycjonowanie i paralelizm zapytań. Bezpieczeństwo i dostęp: Definiuje podstawowe role. Uwierzytelnianie oparte na wtyczkach (np. PAM). Role od wersji 8.0. Mniej szczegółowe uprawnienia. Zaawansowane role, zabezpieczenia na poziomie wiersza, solidne uwierzytelnianie (np. SSL, Kerberos). Bardzo bezpieczne dla środowisk korporacyjnych. Rozszerzenia i personalizacja: Nie dotyczy bezpośrednio. Ograniczone; wtyczki, takie jak do silników pamięci masowej. Duża społeczność, ale kontrolowana przez Oracle. Wysoka rozszerzalność: tworzy typy, funkcje i operatory w C/Python itp. Rozszerzenia, takie jak TimescaleDB dla danych szeregów czasowych. Licencja i koszt: Nie dotyczy (standard otwarty). Oprogramowanie open source (GPL), ale płatna wersja Enterprise. Bezpłatne do podstawowego użytkowania. W pełni open source (licencja PostgreSQL, podobna do BSD). Zawsze bezpłatne; wsparcie dla przedsiębiorstw za pośrednictwem stron trzecich. Typowe zastosowania: Podstawa dla wszystkich systemów RDBMS. Aplikacje internetowe (WordPress, Drupal), e-commerce (Magento), startupy dla uproszczenia. Aplikacje korporacyjne, GIS (PostGIS), hurtownie danych, aplikacje ze złożonymi danymi (np. z dużą ilością JSON). Ograniczenia: Nie jest produktem; zależy od implementacji. Mniej rygorystyczne (może prowadzić do subtelnych błędów).

Wsparcie Late CHECK. Mapa drogowa Oracle Controls. Bardziej stroma krzywa uczenia się; zużywa więcej zasobów w małych konfiguracjach. Społeczność i wsparcie: Standard ISO. Duża społeczność; fora takie jak Stack Overflow. Wsparcie Oracle Enterprise. Silna społeczność i akademickie; częste aktualizacje. Doskonała dokumentacja.

Podsumowanie kluczowych różnic

SQL vs. MySQL/PostgreSQL: SQL to wspólny „język”; MySQL i PostgreSQL to „dialekty”, które posługują się nim z różnymi akcentami. Oba systemy używają SQL, ale dodają/usuwają funkcje. Na przykład standardowe zapytanie SQL, takie jak SELECT * FROM table WHERE id = 1, działa w obu systemach, ale rozszerzenia, takie jak JSON_EXTRACT w MySQL i ->> w PostgreSQL, różnią się.

MySQL kontra PostgreSQL: MySQL jest łatwiejszy i szybszy dla początkujących i prostych aplikacji internetowych, ale mniej rygorystyczny i ma historyczne ograniczenia (np. CHECK). PostgreSQL jest bardziej wydajny i ustandaryzowany w przypadku złożonych aplikacji, oferując lepszą obsługę danych niestrukturalnych i lepsze bezpieczeństwo. Wybierz MySQL ze względu na szybkość odczytu; PostgreSQL ze względu na solidność i skalowalność analityczną.

Aktualne trendy (2026): MySQL nadal dominuje w internecie (dzięki kompatybilności z PHP/Node.js), podczas gdy PostgreSQL zyskuje popularność w chmurze (np. AWS RDS, Google Cloud SQL) dzięki takim funkcjom jak wyszukiwanie wektorowe w sztucznej inteligencji. Oba są darmowe i dojrzałe; wypróbuj oba na swoim kursie, aby je porównać.

Machine Translated by Google

Moduł 2: Model relacyjny

Model relacyjny stanowi teoretyczną podstawę relacyjnych baz danych, takich jak MySQL i MariaDB. Został zaproponowany przez Edgara F. Codda w 1970 roku i przedstawia dane w postaci tabel. połączone logicznymi relacjami. Ten model zapewnia spójność, integralność i wydajność przechowywania i wyszukiwania danych. Zbadamy
jego podstawowe składniki, kluczowe koncepcje i mechanizmy służące zachowaniu integralności danych.

W celu zilustrowania omawianych koncepcji posłużmy się praktycznymi przykładami języka SQL, kompatybilnymi z MySQL/MariaDB.

1. Składniki modelu relacyjnego: tabele, wiersze i kolumny

Model relacyjny organizuje dane w dwuwymiarowe struktury nazywane tabelami (znanymi również jako relacje).

Każda tabela reprezentuje jednostkę lub koncepcję ze świata rzeczywistego, np. „Klienci” lub „Produkty”.

Tabele

Są to zbiory danych zorganizowane według ustalonego schematu. Każda tabela ma unikalną nazwę i definiuje strukturę przechowywanych w niej danych.

Wydziwianie

Reprezentują one poszczególne wystąpienia encji. Każdy wiersz to kompletny zapis danych dla konkretnej encji (na przykład konkretnego klienta).

Kolumny

Definiują pola lub właściwości encji. Każda kolumna ma nazwę, typ danych i ewentualnie ograniczenia.

Przykład wizualny: Wyobraźmy sobie tabelę o nazwie „Pracownicy”

ID	nazwa	nazwisko	dział
1	Wola	Kowal	Obroty
2	znaki	Zuckerberg	TY

W tym przykładzie:

Tabela nosi nazwę „Pracownicy”.

Są 2 wiersze (rekordów pracowników).

Istnieją 4 kolumny (atrybuty takie jak ID, Nazwa itp.).

Machine Translated by Google

W SQL (MySQL/MariaDB) utworzylibyśmy tę tabelę w następujący sposób:

```
UTWÓRZ TABELĘ Pracownicy (
        Identyfikator INT,
        Imię VARCHAR(50),
        Nazwisko VARCHAR(50),
        Dział VARCHAR(50)
);
```

2. Podstawowe koncepcje: krotki, atrybuty, domeny

Terminy te wywodzą się z teorii relacji i odpowiadają praktycznym składnikom wymienionym powyżej, jednak stosują bardziej formalne podejście.

Krotki
Oto wiersze tabeli. Krotka to uporządkowany zbiór wartości reprezentujących unikatową instancję. W poprzednim przykładzie (1, Juan, Pérez, Sales) jest krotką. Krotki muszą być unikatowe w połączeniu z kluczami (patrz poniżej).

Atrybuty
Oto kolumny tabeli. Każdy atrybut opisuje właściwość encji i ma unikalną nazwę w tabeli. Na przykład „Nazwa” jest atrybutem.

Domeny
Definiują zbiór dozwolonych wartości dla atrybutu. To jak „typ danych” z ograniczeniami.

Na przykład:

Domena „Wiek”: Liczby całkowite dodatnie pomiędzy 18 i 65.

Domena „E-mail”: ciągi znaków odpowiadające wzorcowi wiadomości e-mail.

W języku SQL domeny są implementowane przy użyciu typów danych i ograniczeń (patrz sekcja 5).

Machine Translated by Google

Przykład:

```
UTWÓRZ TABELĘ Ludzie (
        ID INT, -- Domena: Liczby całkowite
        Wiek INT CHECK (Wiek 18 I Wiek 65), -- Domena zastrzeżona
        E-mail VARCHAR(100) — Domena: ciągi znaków do 100 znaków
);
```

3. Klucze podstawowe i klucze obce

Klucze to atrybuty lub kombinacje atrybutów, które jednoznacznie identyfikują krotki i ustanawiają relacje między tabelami.

Klucz podstawowy

Jednoznacznie identyfikuje każdą krotkę w tabeli. Musi być unikatowa i różna od null.
Zwykle jest to automatycznie zwiększający się identyfikator.

Właściwości: unikalne, różne od null, automatycznie indeksowane w celu szybkiego wyszukiwania.

Przykład: W tabeli „Pracownicy" „ID" jest kluczem podstawowym.

Klucz obcy

Odwołanie do klucza podstawowego innej tabeli, ustanawiające relację. Zapewnia, że wartości w tej kolumnie istnieją w tabeli, do której się odwołuje.

Przykład w SQL:

```
UTWÓRZ TABELĘ Działy (
        ID_Department INT PRIMARY KEY, -- Klucz podstawowy
        Nazwa VARCHAR(50)
);

UTWÓRZ TABELĘ Pracownicy (
        ID_Empleado INT PRIMARY KEY, -- Klucz podstawowy
        Imię VARCHAR(50),
```

Machine Translated by Google

```
9          ID_Dział INT,

10         ODNIESIENIA DO KLUCZA OBCEGO (ID_DZIAŁU)
Działy (ID_Dział) — klucz obcy

11 );
```

Tutaj „ID_Departamento” w „Empleados” musi odpowiadać istniejącej wartości w „Departamentos”.

4. Typy relacji: Jeden do jednego (1:1), Jeden do wielu (1:N), Wiele do wielu (N:M)

Relacje określają sposób, w jaki jednostki są połączone poprzez klucze obce.

Są one modelowane na diagramach ER (Entity-Relationship) i implementowane w tabelach.

Jeden na jednego (1:1)
Każde wystąpienie jednego bytu jest powiązane z dokładnie jednym wystąpieniem innego bytu.

Na przykład pracownik ma jeden profil użytkownika.

Implementacja: Umieść klucz obcy w jednej z tabel.

```
1 UTWÓRZ TABELĘ Pracownicy (

2          ID INT KLUCZ PODSTAWOWY,

3          Nazwa VARCHAR(50)

4 );

5

6 UTWÓRZ PROFILE TABEL (

7          ID_Pracownika INT PRIMARY KEY,

8          Użytkownik VARCHAR(50),

9          KLUCZ OBCY (ID_Pracownik) REFERENCJE Pracownicy (ID)

10 );
```

Jeden do wielu (1:N)
Jedno wystąpienie jednego bytu jest powiązane z wieloma wystąpieniami innego bytu, ale nie odwrotnie. Przykład: W dziale pracuje wielu pracowników.

Realizacja
FK w tabeli „wielu” (Pracownicy).
SQL: Podobnie jak w przykładzie klucza obcego powyżej.

Machine Translated by Google

Wiele do wielu (N:M)
Wiele wystąpień jednego bytu jest powiązanych z wieloma wystąpieniami innego.
Przykład: Studenci i kursy (student może uczęszczać na wiele kursów, kurs ma wielu studentów).

Realizacja
Wymaga tabeli pośredniej (łączącej) z kluczem obcym do obu tabel.

```
UTWÓRZ TABELĘ Studenci (
        ID INT KLUCZ PODSTAWOWY,
        Nazwa VARCHAR(50)
);

Kursy CREATE TABLE (
        ID INT KLUCZ PODSTAWOWY,
        Nazwa VARCHAR(50)
);

CREATE TABLE Rejestracje (
        Identyfikator_studenta INT,
        ID_Kurs INT,
        KLUCZ PODSTAWOWY (ID_Estudiante, ID_Curso), -- Złożony klucz podstawowy

        KLUCZ OBCY (ID_Estudiante) REFERENCJE Studenci(ID),
        KLUCZ OBCY (ID_KURSU) REFERENCJE Kursy (ID)
);
```

5. Integralność referencyjna i ograniczenia (UNIQUE, NOT NULL, CHECK)

Integralność gwarantuje spójność i poprawność danych. MySQL/MariaDB obsługuje kilka ograniczeń.

Machine Translated by Google

Integralność referencyjna
Gwarantowane przez KLUCZE OBCE. Zapobiega powstawaniu osieroconych wartości (np. pracownik nie może odwołać się do nieistniejącego działu).

Opcje
ON DELETE CASCADE (usuwa zależności), ON UPDATE RESTRICT (zapobiega zmianom, które powodują przerwanie odwołań).

Ograniczenia

UNIKALNY
Upewnij się, że wartości w kolumnie (lub kombinacji) są unikalne. Np.: Unikalne adresy e-mail.

NIE NULL
Zapobiega wyświetlaniu wartości null w kolumnie. Np.: Wymagane nazwy.

SPRAWDZAĆ
Zaznacz niestandardowy warunek. Np.: Wiek > 18.

```
UTWÓRZ TABELĘ Użytkownicy (
        ID INT AUTO_INCREMENT KLUCZA PODSTAWOWEGO ,
        E-mail VARCHAR(100) UNIQUE NOT NULL, -- Unikalny i niepusty
        Wiek INT CHECK (wiek     18), -- Stan
        ID_Perfil INT,
        ODNIESIENIA DO KLUCZA OBCEGO (ID_PROFILU) Profile(ID) PRZY USUWANIU KASKADOWYM
-- Integralność referencyjna
);
```

Informacje o ON DELETE i ON UPDATE

Opcje te definiuje się podczas tworzenia klucza obcego. Kontrolują one działania wykonywane automatycznie w przypadku próby modyfikacji lub usunięcia danych w tabeli nadrzędnej (tej z KLUCZEM PODSTAWOWYM).

1. ON DELETE CASCADE „Usuń osoby zależne” (automatyczne propagowanie usuwania)

Machine Translated by Google

Po usunięciu wiersza w tabeli nadrzędnej moduł bazy danych automatycznie usuwa wszystkie wiersze w tabelach podrzędnych, które odwoływały się do tego wiersza.
REMOVED.

proste tłumaczenie
„Jeśli zabijesz ojca, zabij też wszystkie jego dzieci".

Kiedy jest używane (bardzo powszechne):

Dane zależne, które nie mają sensu bez danych nadrzędnych komentarze do wpisu, elementy w zamówieniu, zdjęcia użytkownika, sesje logowania itp.

Chcesz automatycznego czyszczenia i uniknąć osieroconych rekordów.

Przykład praktyczny (system blogowy):

```
UTWÓRZ TABELĘ postów (
        id INT AUTO_INCREMENT KLUCZA PODSTAWOWEGO ,
        tytuł VARCHAR(200) NOT NULL
);

UTWÓRZ TABELĘ komentarze (
        id INT AUTO_INCREMENT KLUCZA PODSTAWOWEGO ,
        post_id INT NOT NULL,
        TEKST tekst,
        KLUCZ OBCY (post_id) ODNIESIENIA posty (id)
                USUWANIE KASKADOWE          Oto jest!
                OGRANICZENIE AKTUALIZACJI
);
```

Machine Translated by Google

Dowód:

```
INSERT INTO posts (title) VALUES ('Mój pierwszy post');

INSERT INTO comments (post_id, text) VALUES (1, 'Świetnie!'), (1, 'Podobało mi się');

USUŃ Z postów GDZIE id = 1;
```

Wynik usuwa post o identyfikatorze 1

Wszystkie komentarze pod tym wpisem zostaną automatycznie usunięte.
Nie ma żadnych osieroconych komentarzy wskazujących na post, który już nie istnieje.

Zalety
Automatyczne czyszczenie, mniej kodu w aplikacji.

Niebezpieczeństwo
To destrukcyjne! Jeśli przypadkowo usuniesz ważny rekord, usuniesz wszystko, co od niego zależy (uważaj na produkcji!).

2. OGRANICZENIE AKTUALIZACJI „Zapobiega zmianom, które powodują przerwanie odniesień”

Podczas próby modyfikacji (AKTUALIZACJI) wartości klucza podstawowego (lub unikatowego) w tabeli nadrzędnej silnik blokuje operację, jeśli w tabeli podrzędnej znajduje się co najmniej jeden wiersz odwołujący się do niego.

Proste tłumaczenie:
„Jeśli są dzieci nie można zmienić dowodu osobistego ojca!”

Jest to wartość domyślna (wraz z ON DELETE RESTRICT).

Przykład z tym samym schematem co powyżej:

```
AKTUALIZACJA postów USTAW id = 100 GDZIE id = 1;
```

Wynik Błąd podobny do:

```
BŁĄD 1451 (23000): Nie można usunąć ani zaktualizować wiersza nadrzędnego: ograniczenie klucza obcego nie powiodło się

(`blog`.`comments`, OGRANICZENIE `comments_ibfk_1` KLUCZ OBCY (`post_id`) ODNIESIENIA `posts` (`id`))
```

Machine Translated by Google

Nie możesz zmienić identyfikatora wpisu, ponieważ istnieją komentarze odnoszące się do niego.
Kiedy należy z niej korzystać (zalecane prawie zawsze w przypadku kluczy podstawowych):

- Kiedy klucz podstawowy nigdy nie powinien się zmieniać (automatyczne zwiększanie identyfikatorów, unikatowe kody itp.).
- Chroni integralność: zapobiega przypadkowemu uszkodzeniu tysięcy odniesień.
- Bardzo przydatne w tabelach, w których ID jest stałe (użytkownicy, produkty, kategorie).

Szybkie porównanie najpopularniejszych opcji

Działanie	PRZY USUNIĘCIU	AKTUALIZACJA	Kiedy go używać	Poziom "agresywność"
OGRANICZ	Zapobiega usunięciu	Zapobiega aktualizacji	Ważne dane, chcesz mieć kontrolę ręczną	Bezpieczny / Defensywny
Usuwanie łańcucha	CASCADE	Aktualizacja na temat łańcuch	Dane zależne (logi, szczegóły, elementy)	Automatycznie / Niebezpieczny
USTAW NULL	Wstawia NULL syn	Wstawia NULL syn	Syn może żyć bez ojca (np. opcjonalnie autor)	Mediator
BRAK DZIAŁANIA	Tak jak OGRANICZAĆ	Tak jak OGRANICZAĆ	Prawie identyczny z RESTRICT (standard SQL)	Jasne

Typowe zalecenia w większości projektów:

KLUCZ OBCY (...) ODNIESIENIA ...

OGRANICZ USUWANIE -- lub CASCADE, jeśli jest to bezpieczne

OGRANICZENIE AKTUALIZACJI – prawie zawsze OGRANICZENIE

Podsumowanie wizualne (analogia)

PO USUNIĘCIU KASKADY Jak w nawiedzonym domu: jeśli zniszczysz ducha-bohatera (ojca), wszystkie pomniejsze duchy automatycznie znikną.

•

OGRANICZENIE AKTUALIZACJI Jak blokada na dowodzie osobistym: jeśli ktoś ma dzieci zarejestrowane na Twoim dowodzie osobistym, nie możesz tego zmienić! Musisz je najpierw „odłączyć” (usunąć lub zaktualizować dane dzieci).

Machine Translated by Google

Moduł 3: Koncepcyjne projektowanie baz danych

Projekt koncepcyjny to pierwszy etap rozwoju relacyjnej bazy danych, w którym abstrakcyjnie modeluje się świat rzeczywisty, nie przejmując się szczegółami technicznymi, takimi jak typy danych czy indeksy.

Ten moduł skupia się na przechwytywaniu wymagań użytkowników i przedstawianiu ich za pomocą modelu encji-związków (modelu ER), zaproponowanego przez Petera Chena w 1976 r. Model ER pomaga wizualizować encje, ich właściwości i wzajemne powiązania, ułatwiając przejście do modelu relacyjnego (jak w MySQL/MariaDB).

Aby zilustrować omawiane koncepcje, posłużmy się praktycznym przykładem „biblioteki" z poprzedniego modułu, gdyż jest to przypadek realistyczny i skalowalny.

1. Analiza wymagań systemowych

Analiza wymagań to proces gromadzenia, dokumentowania i walidacji potrzeb systemu informatycznego.
Jest ona kluczowa dla uniknięcia kosztownych błędów w późniejszych fazach. Dzieli się na etapy: pozyskiwanie (wywiady, ankiety), analiza (identyfikacja niespójności), specyfikacja (dokumentowanie) i walidacja (potwierdzenie z interesariuszami).

Kluczowe kroki: Identyfikacja obiektów i relacji: Zastanów się, jakie „rzeczy" (obiekty) znajdują się w systemie i w jaki sposób są one ze sobą powiązane.

Zdefiniuj atrybuty: Dla każdej jednostki wypisz właściwości (np. nazwę, datę).
Weź pod uwagę zasady biznesowe: ograniczenia, takie jak „książki nie można wypożyczyć, jeśli zostanie zgubiona".

Narzędzia: Wykorzystaj przypadki użycia, schematy blokowe i historie użytkowników.

Przykład w bibliotece:

Żądane wymagania: „Biblioteka musi rejestrować książki, autorów, egzemplarze fizyczne, użytkowników i wypożyczenia. Użytkownik może wypożyczyć maksymalnie 3 książki na raz; wypożyczenie trwa 15 dni".

Analiza: Zidentyfikowaliśmy takie jednostki, jak Książka, Autor, Użytkownik; relacje, takie jak „autor pisze książkę"; ograniczenia, takie jak „kopia musi istnieć, aby można ją było wypożyczyć".

Wynik: Dokument wymagań stanowiący podstawę dla MER.

Machine Translated by Google

2. Model relacji encji (ER): encje, atrybuty i relacje

Model ER przedstawia dziedzinę problemu za pomocą trzech głównych komponentów:

Encje: Reprezentują one obiekty lub koncepcje ze świata rzeczywistego, które istnieją niezależnie. Mogą być namacalne (np. Książka) lub abstrakcyjne (np. Pożyczka). Każda encja ma nazwę (w liczbie pojedynczej, pisaną wielkimi literami) i jest reprezentowana przez prostokąt.

Atrybuty: Opisowe właściwości jednostek. Typy:

Prosty: pojedyncza wartość (np. tytuł książki).

Związki: podzielone na podatrybuty (np. imię i nazwisko = imię + nazwisko).

Wielowartościowy: Może mieć kilka wartości (np. gatunki książki, reprezentowane za pomocą nawiasów klamrowych {}).

Pochodne: Obliczane na podstawie innych (np. wiek uzyskany z daty urodzenia, linią przerywaną).

Klucz: Atrybut(y) jednoznacznie identyfikujący(e) jednostkę (podkreślone).

Relacje: Połączenia między encjami. Reprezentowane jako diamenty. Mogą mieć własne atrybuty (np. datę w relacji „pożycza").

Przykład w bibliotece:

- Jednostka: Książka (atrybuty: ISBN (klucz), tytuł, rok_publikacji).
- Relacja: Napisz (pomiędzy Autorem a Książką) z atrybutem „year_collaboration", jeśli ma to zastosowanie.

3. Diagramy ER: notacja i symbolika

Diagramy ER wykorzystują standardowe notacje, takie jak notacja Chena lub „kurzej łapki", dla większej przejrzystości. Notacja Chena jest nauczana na kursach ze względu na swoją prostotę.

Podstawowe symbole (notacja Chena):

Silny obiekt: Prostokąt (np. [Książka]).

Słaby obiekt: prostokąt z podwójną ramką (np. [Przykład]).

Atrybut: Owal połączony z bytem (podkreślony, jeśli jest kluczem).

Relacja: Romb połączony liniami z obiektami.

Atrybut wielowartościowy: Owal z podwójną obwódką.

Machine Translated by Google

Relacja rekurencyjna: Romb połączony z tym samym bytem (np. „is_a_sequel_to” w książkach).

Notacja Crow's Foot (popularna alternatywa):

Linie kończą się „odgałęzieniami” oznaczającymi kardynalność: | dla 1, > dla wielu.

Przykład: Encja1 —|—> Encja2 (od 1 do wielu).

Uproszczony przykład tekstowy diagramu ER dla biblioteki (z wykorzystaniem notacji opisowej):

```
[Autor] ———— (pisze) ———— [Książka] ———— (ma) ———— [Kopia (słaby)]
                                          |
| (atrybuty: id_autor,                    (atrybuty: isbn, tytuł)
  imię, narodowość)
```

4. Kardynalność i uczestnictwo w relacjach

Kardynalność: wskazuje, ile wystąpień jednego bytu jest powiązanych z wystąpieniami innego. Typy:

1:1 (jeden do jednego): Np. jeden pracownik ma jedno biurko.

1:N (jeden do wielu): Np. książka ma wiele egzemplarzy.

N:M (wiele do wielu): Np. autorzy piszą wiele książek i odwrotnie.

N:1 (wiele do jednego): Odwrotność 1:N.

Uczestnictwo: Określa, czy relacja jest obowiązkowa czy opcjonalna.

Razem (obowiązkowe): Podwójna linia lub wypełnione kółko (wszystkie wystąpienia muszą brać udział, np. kopia musi należeć do książki).

Częściowo (opcjonalnie): prosta linia lub puste koło (niektóre wystąpienia mogą nie brać udziału, np. książka może nie mieć żadnych aktywnych wypożyczeń).

Na diagramach: Kardynalność oznaczana jest w pobliżu jednostki (np. 1 lub N), a udział ten zależy od grubości linii.

Przykład w bibliotece:

relacja „ma” (książka-kopia): 1:N, całkowity udział w kopii (kopia zawsze należy do książki), częściowy udział w książce (książka może nie mieć kopii).

Machine Translated by Google

5. Słabe i silne podmioty

Silne jednostki: mają niezależny byt i własny klucz podstawowy (np. książka z unikalnym numerem ISBN). Są reprezentowane jako proste prostokąty.

Słabe encje: Ich istnienie i identyfikacja zależą od silnej encji. Nie posiadają one własnego, kompletnego klucza podstawowego; wykorzystują klucz częściowy oraz klucz silnej encji (klucz różniczkowy). Są reprezentowane przez podwójny prostokąt.

Identyfikacja relacji: Połączenie ze stroną silną jest rombem z podwójną obwódką, przy całkowitym udziale strony słabej.

Przykład w bibliotece:

Mocne: Książka (klucz: ISBN).
Słabe: Kopia (częściowy klucz: inventory_num; zależy od numeru ISBN książki). Relacja „ma” jest identyfikatorem.

Machine Translated by Google

Moduł 4: Projektowanie logiczne i standaryzacja

Wstęp

Projektowanie logiczne przekształca model koncepcyjny (ER/ER) w schemat relacyjny, który można wdrożyć w systemie DBMS, takim jak MySQL/MariaDB. Obejmuje to konwersję do tabel, definicję kluczy i normalizację w celu wyeliminowania redundancji i anomalii.

Normalizacja to systematyczny proces oparty na teorii zależności funkcyjnych, który porządkuje dane w celu:

- Unikaj redundancji
- Zapobiegaj anomaliom wstawiania, aktualizacji i usuwania
- Zapewnij integralność

W praktyce większość baz danych jest normalizowana do standardów 3NF lub BCNF, a czasami stosowana jest denormalizacja w celu kontroli wydajności.

1. Transformacja modelu ER do modelu relacyjnego

Standardowe zasady (oparte na notacji Chena i powszechnych zasadach):

Silne encje Tabele z ich atrybutami. Klucz podstawowy (PK) jest zachowany.

Słabe encje Tabele, których klucz główny jest złożony: klucz częściowy + klucz główny silnej encji, od której zależą.

Relacje 1:1 Umieść FK w jednej z tabel (najlepiej tabeli uczestnictwa) (całkowite lub opcjonalne). Jeśli oba są obowiązkowe, można je połączyć w jedną tabelę.

Relacje 1:N FK w tabeli po stronie „N" (tej, która uczestniczy z wieloma).

Relacje N:M Nowa tabela pośrednia z kluczem obcym do obu encji + złożony klucz podstawowy (lub klucz zastępczy). Atrybuty relacji znajdują się tutaj.

Atrybuty wielowartościowe Nowa tabela z kluczem obcym do encji + wartość wielowartościowa (klucz złożony).

Atrybuty złożone Rozłóż na proste atrybuty lub zachowaj jako kolumnę jeśli system DBMS na to pozwala.

Relacje rekurencyjne Takie same jak relacje binarne, ale FK wskazuje na tę samą tabelę.

Machine Translated by Google

Przykład praktyczny – Biblioteka (z modułu 3):

Główne podmioty i relacje:

Autor (silne) — pisze (N:M) — Książka (silne) — ma (1:N) — Kopia (słaba)

Użytkownik (silny) — wykonuje (1:N) — Pożyczka (silny) — pożycza (N:1) — Kopiuje

Wynikowe tabele (MySQL/MariaDB):

SQL-- Silna jednostka: Autor

```
UTWÓRZ TABELĘ Autorzy (
        author_id INT AUTO_INCREMENT KLUCZA PODSTAWOWEGO ,
        nazwa VARCHAR(100) NOT NULL,
        narodowość VARCHAR(50),
        data_urodzenia DATA
);

-- Silny podmiot: Książka
Książki CREATE TABLE (
        isbn VARCHAR(20) KLUCZ PODSTAWOWY,
        tytuł VARCHAR(200) NOT NULL,
        rok_publikacji ROK,
        liczba_stron INT
);

-- Relacja autor-książka N:M     tabela pośrednia
UTWÓRZ TABELĘ Libro_Autores (
        isbn VARCHAR(20),
        id_autor INT,
        KLUCZ PODSTAWOWY (ISBN, id_autor),
        KLUCZ OBCY (ISBN) ODNIESIENIA Książki (ISBN) NA USUNIĘCIE KASKADOWE,
```

Machine Translated by Google

```
22          KLUCZ OBCY (author_id) ODNIESIENIA Autorzy (author_id) PRZY USUNIĘCIU OGRANICZAĆ
23 );
24
25 -- Słaby podmiot: Kopia (zależy od książki)
26 Przykłady CREATE TABLE (
27          isbn VARCHAR(20),
28          num_inventario INT,
29 stan ENUM('dostępny', 'pożyczony', 'utracony') DOMYŚLNY 'dostępny',
30          lokalizacja VARCHAR(20),
31          KLUCZ PODSTAWOWY (ISBN, numer_inwentarza),
32          KLUCZ OBCY (ISBN) ODNIESIENIA Książki (ISBN) USUWANE KASKADOWO
33 );
34
35 -- Silny podmiot: Użytkownik
36 UTWÓRZ TABELĘ Użytkownicy (
37          dni VARCHAR(12) KLUCZ PODSTAWOWY,
38          nazwa VARCHAR(100) NOT NULL,
39          e-mail VARCHAR(100) UNIQUE NOT NULL
40 );
41
42 -- Podmiot pożyczki (z kluczem pierwotnym dla użytkownika i kopią)
43 UTWÓRZ TABELĘ Pożyczki (
44          loan_id INT AUTO_INCREMENT KLUCZA PODSTAWOWEGO ,
45          user_id VARCHAR(12) NOT NULL,
46          isbn VARCHAR(20) NOT NULL,
47          num_inventario INT NOT NULL,
48          loan_date DATA NIE NULL,
49          data_zwrotu DATA NULL,
50          KLUCZ OBCY (dni_user) REFERENCJE Użytkownicy (dni),
```

Machine Translated by Google

```
51          KLUCZ OBCY (isbn, inventory_num) ODNIESIENIA Egzemplarze (isbn,
numer_inwentarza)

52 );
```

2. Zależności funkcjonalne

Zależność funkcjonalna (FD) X Y oznacza, że wartość X jednoznacznie determinuje wartość Y.

Klucz kandydata: minimalny zbiór atrybutów, który determinuje wszystkie pozostałe (może być ich kilka).

Częściowa zależność: atrybut niebędący kluczem zależy tylko od części klucza złożonego.

Zależność przechodnia: atrybut niekluczowy zależy od innego atrybutu niekluczowego (A B C).

Przykład w nienormalizowanej tabeli „BookLoans”:

ISBN	kwalifikacja	autor	dni_użytkownika	nazwa użytkownika	loan_date
123	Don Kichot	Cervantesa	12345678A	Ana Pérez	2025-01-10
123	Don Kichot	Cervantesa	87654321B	Juan Gómez	2025-02-15

Działy:

isbn tytuł, autor (częściowo, jeśli PK to isbn + user_id)

user_ID user_name (przechodnie, jeśli ISBN coś określa)

3. Postacie normalne: 1NF, 2NF, 3NF i BCNF (Boyce-Codd)

1NF (pierwsza postać normalna): Wszystkie atrybuty są atomowe (bez wielu wartości ani powtarzających się grup). Każda komórka ma tylko jedną wartość. Przykład nie w 1NF: kolumna „autorzy” = „Cervantes, Quevedo” rozdzielenie na tabelę pośrednią.

2NF: Spełnia 1NF + brak zależności częściowych. Każdy atrybut niebędący kluczem jest w pełni zależny od klucza podstawowego. Przykład: W tabeli z kluczami podstawowymi (ISBN, user_id), jeśli „tytuł” zależy tylko od numeru ISBN anomalia. Rozwiązanie: oddzielenie książek i wypożyczeń.

3NF: Spełnia 2NF + brak zależności przechodnich. Atrybuty niekluczowe zależą tylko od klucza kandydującego, a nie od innych atrybutów niekluczowych. Przykład: Jeśli dni_usuario nombre_usuario i nombre_usuario email anomalia. Rozwiązanie: oddzielna tabela „Użytkownicy”.

FNBC (Boyce-Codd): Zgodny z 3NF + każdy wyznacznik jest kluczem kandydującym (bardziej rygorystyczny niż 3NF). Eliminuje anomalie w przypadku wielu kluczy kandydujących.

Machine Translated by Google

Przykład: Tabela Profesor-Przedmiot-Sala lekcyjna, gdzie Profesor Sala lekcyjna (ale PK to Profesor+Temat) naruszają BCNF. Rozwiązanie: rozdzielenie.

4. Proces normalizacji i denormalizacji

Typowy proces:

Zacznij od tabel początkowych (czasami nie znormalizowanych).

Przenieś do 1NF usuń grupy wielowartościowe/powtarzające się.

Aby przejść do 2NF wyeliminuj częściowe zależności (podziel tabele).

Doprowadzenie do 3NF eliminacja zależności przechodnich.

Sprawdź BCNF pod kątem nakładających się kluczy kandydackich.

Kompletny przykład normalizacji (początkowa nienormalizowana tabela biblioteki):

PrestamoID	ISBN	Kwalifikacja	Autor	Numer identyfikacyjny użytkownika	Nazwa użytkownika	Data pożyczki	Data zwrotu
1	123	Don Kichot	Cervantesa 111		Anna	2025-01-01	NIEWAŻNY

1FN: OK (atomowy).
2FN: Książki oddzielnie (ISBN tytuł, autor) i wypożyczenia.
3FN: Oddziel autorów, jeśli N:M, użytkownicy (dni nazwa).

Denormalizacja:

Robi się to celowo, aby poprawić wydajność (mniej połączeń JOIN) w systemach OLAP lub systemach intensywnie wykorzystujących odczyt.

Przykład:

Dodaj „nazwę użytkownika” do sekcji Pożyczki, aby uniknąć konieczności dołączania użytkowników w często zadawanych pytaniach.

Ryzyko: Nadmiarowość anomalie aktualizacji. Użyj wyzwalaczy lub widoków, aby zachować spójność.

5. Projektowanie efektywnych schematów

Normalizacja do 3FN/BCNF dla OLTP (transakcje).

Wybiórcza denormalizacja na potrzeby raportowania (dodanie obliczonych kolumn, tabel podsumowujących).

Użyj indeksów w kolumnach FK i częstych w WHERE.
Należy rozważyć partycjonowanie i materializowanie widoków w dużych bazach danych.

Machine Translated by Google

05.- Język definicji danych (DDL)

DDL jest „architektem” świata SQL; tutaj definiujemy strukturę, a nie dane. To jak projektowanie planu domu przed położeniem cegieł. Jeśli zostanie to zrobione poprawnie,
Reszta projektu będzie łatwa i szybka. Jeśli jednak będzie słabo zdefiniowana... będziemy mieli mnóstwo problemów z rozwiązywaniem problemów.

Omówimy każdy temat jasno, szczegółowo i w 100% praktycznie, zawsze koncentrując się na MySQL (wersja 8.0+, która jest obecnie najpowszechniej używaną wersją). Będziemy korzystać z przykładów z życia wziętych, które możesz skopiować i wkleić do swojego MySQL Workbench lub wiersza poleceń.

1. Wprowadzenie do standardowego SQL

SQL to skrót od Structured Query Language (SJL) . Powstał w latach 70. XX wieku w IBM pod nazwą SIQUEL i został ustandaryzowany w 1986 roku przez ANSI (American National Standards Institute). Obecnie obowiązuje standard SQL:2016 (i jego fragmenty).
(2023)

Jednak żaden silnik bazy danych nie jest w 100% zgodny ze standardem. Każdy ma swój własny „dialekt”:

Silnik	Stopień zgodności z normą	Ważne cechy
MySQL / MariaDB	Bardzo wysoki (95%)	Bardzo tolerancyjny, szybki czytelnik
PostgreSQL	Najsurowsze (98%)	Bardziej wierny standardowi
Serwer SQL	Wysoki	T-SQL (zastrzeżony)
Wyrocznia	Wysoki	PL/SQL

W MySQL DDL jest szczególnie prosty i intuicyjny. Dlatego jest chętnie używany do nauki i projektów internetowych (WordPress, Laravel, Django itp.).

Wyobraź sobie DDL jako „umowę prawną” dla naszej bazy danych. Po jej podpisaniu (wykonaniu) całe oprogramowanie korzystające z tej bazy danych musi ją respektować. Dlatego musisz bardzo dokładnie przemyśleć, zanim napiszesz pierwszą linijkę.

2. UTWÓRZ BAZĘ DANYCH – Tworzenie bazy danych

Pierwsze zlecenie, jakie zawsze realizujemy.

Podstawowa składnia MySQL:

```
UTWÓRZ BAZĘ DANYCH nazwa_bazy_danych
        ZESTAW ZNAKÓW utf8mb4
```

Machine Translated by Google

```
3            Sortuj utf8mb4_unicode_ci;
```

Dlaczego utf8mb4?

Ponieważ obsługuje emoji, chiński, arabski, matematykę... Stare kodowanie UTF-8 MySQL pozwalało na zapisywanie danych tylko do 3 bajtów i nie działało przy wielu znakach. Od wersji MySQL 5.5.3 używanie UTF-8MB4 jest obowiązkowe.

Zalecane dobre praktyki:

SQL – Profesjonalny sposób (z zabezpieczeniami)

```
1 UTWÓRZ BAZĘ DANYCH, JEŚLI NIE ISTNIEJE online_store
2            ZESTAW ZNAKÓW utf8mb4
3            Sortuj utf8mb4_unicode_ci;
```

Aby wyświetlić wszystkie bazy danych na naszym serwerze , używamy polecenia

```
1 POKAŻ BAZY DANYCH;
```

Wybierz ten, którego będziemy używać (bardzo ważne!)

```
1 UŻYJ sklepu internetowego;
```

Usuń (ostrożnie, nie ma odwrotu)

```
1 USUŃ BAZĘ DANYCH, JEŚLI ISTNIEJE online_store;
```

> Nigdy nie używaj polecenia CREATE DATABASE bez polecenia IF NOT EXISTS w skryptach migracji.

Unikniesz błędów przy ponownym uruchomieniu skryptu.

3. Tworzenie i modyfikowanie tabel: CREATE TABLE, ALTER TABLE, DROP TABLE

To tutaj zaczyna się prawdziwa zabawa.

CREATE TABLE – klejnot języka DDL

```
1 CREATE TABLE klienci (
2            id                    BIGINT AUTO_INCREMENT KLUCZ PODSTAWOWY,
3            e-mail                VARCHAR(255) NIE NULL UNIKALNY,
```

Machine Translated by Google

```
4        nazwa                    VARCHAR(100) NIE NULL,
5        data_urodzenia DATA,
6        balansować               DECIMAL(12,2) DOMYŚLNIE 0,00,
7        zaleta                   DOMYŚLNA WARTOŚĆ BOOLE'A PRAWDA ,
8        data_rekordu DATA_CZAS DOMYŚLNA AKTUALNY_ZNACZNIK_CZASU,
9        INDEX idx_name (nazwa)                                  -- indeks normalny
10 );
```

Wyjaśnienie wiersz po wierszu (dydaktyczne):

BIGINT AUTO_INCREMENT: Automatyczna liczba, która nigdy się nie powtarza (idealne dla identyfikatora).

NOT NULL: wymusza, aby zawsze istniała wartość (unika niekompletnych danych).

UNIQUE: nie pozwala na duplikowanie danych, na przykład adresu e-mail lub numeru dowodu osobistego.

DOMYŚLNA: wartość automatyczna, jeśli nic nie zostanie przekazane.

CURRENT_TIMESTAMP: przechowuje dokładną datę i godzinę utworzenia.

ALTER TABLE – „Chirurgia” stołów

Po utworzeniu tabeli możemy ją modyfikować bez utraty danych (prawie zawsze).

SQL-- Dodaj kolumnę

```
1 Klienci ALTER TABLE
2         DODAJ KOLUMNĘ telefono VARCHAR(20) PO nombre;
```

-- Modyfikuj kolumnę (zmień typ lub ograniczenia)

```
1 Klienci ALTER TABLE
2         MODYFIKUJ KOLUMNĘ saldo DECIMAL(15,2) NOT NULL;
```

-- Zmień nazwę kolumny

```
1 Klienci ALTER TABLE
2         ZMIEŃ KOLUMNĘ data_urodzenia data_urodzenia DATA;
```

-- Dodaj indeks po utworzeniu tabeli

```
1 Klienci ALTER TABLE
2         DODAJ INDEKS idx_email (email);3
```

-- Usuń kolumnę (uwaga! utrata danych)

```
1 Klienci ALTER TABLE
2         OPUSZ KOLUMNĘ telefon;
```

Machine Translated by Google

> W środowisku produkcyjnym nigdy nie używaj polecenia ALTER TABLE w przypadku dużych tabel w godzinach pracy.

(> 1 milion wierszy). To może zablokować tabelę na kilka minut. W tym celu polecam narzędzia takie jak pt-online-schema-change (Percona Toolkit).

~~Dokumentacja narzędzia Percona Toolit~~

USUŃ TABELĘ

```
USUŃ TABELĘ, JEŚLI ISTNIEJĄ klienci;
```

Usuń wiele tabel jednocześnie

```
USUŃ TABELĘ JEŚLI ISTNIEJĄ klienci, zamówienia, produkty;
```

4. Typy danych w MySQL (najczęściej używane)

Przyjrzyjmy się im według kategorii, podając prawdziwe przykłady i rekomendacje:
Liczbowy

TINYINT od -128 do 127 (lub 0-255, jeśli UNSIGNED). Idealne dla stanów (0/1).

INT do ±2,147 mln. Najczęściej używane w przypadku normalnych dowodów osobistych.

BIGINT do 9 kwintylionów. Zawsze używaj tej funkcji do identyfikatorów w dużych tabelach.

DECIMAL(10,2) : dokładna kwota. Nigdy nie używaj FLOAT ani DOUBLE w przypadku pieniędzy (błędy zaokrąglanie).

Ciągi tekstowe

CHAR(10) o stałej długości (z dopełnieniem spacjami). Dobry do kodów pocztowych.

VARCHAR(255) zmienna długość. Król pól tekstowych.

TEKST do 65 KB.

ŚREDNITEKST do 16 MB (w przypadku długich artykułów).

DŁUGI TEKST do 4 GB (bardzo rzadko).

Daty i godziny (te, które początkujący najczęściej mylą)

DATA tylko data (2025-02-18)

DATETIME data i godzina (2025-02-18 14:30:25) nie ma strefy czasowej.

Data i godzina TIMESTAMP z automatyczną strefą czasową (zalecane na serwerach).

Boole'a

Machine Translated by Google

MySQL nie ma prawdziwego typu BOOLEAN. Wewnętrznie jest to TINYINT(1):

```
is_active BOOLEAN DEFAULT TRUE — MySQL przekształca go w tinyint i zapisuje
jak 1 lub 0
```

Inne bardzo przydatne

ENUM('pojedynczy', 'żonaty', 'rozwiedziony') Używane w przypadku list z niewielką liczbą opcji

JSON (od MySQL 5.7) zapisuje całe dokumenty bez tworzenia 20 kolumn.

PUNKT, WIELOKĄT do geolokalizacji.

5. Definicja ograniczeń i wskaźników

Ograniczenia to „reguły integralności”, które chronią dane.

Najważniejsze ograniczenia

Ograniczenie	Składnia	Do czego to służy?	Prawdziwy przykład
KLUCZ PODSTAWOWY	KLUCZ PODSTAWOWY (id)	Unikalny identyfikator i automatyczny indeks	ID dowolnej tabeli
UNIKALNY	UNIKALNY (e-mail)	Nie pozwala na duplikaty	e-mail, dni, nazwa użytkownika
NIE NULL	nazwa VARCHAR(100) NOT NULL	Wymagana jest wartość	wymagane pola formularz
KLUCZ OBCY	KLUCZ OBCY (cliente_id) REFERENCJE klienci(id)	Relacja między zamówieniami	tabele klientów
DOMYŚLNY	DOMYŚLNE SALDO DZIESIĘTNE (12,2) 0,00	Wartość domyślna	nowe saldo klienta = 0
SPRAWDŹ (MySQL 8.0+)	SPRAWDŹ (wiek 18)	Minimalny wiek uprawniający do walidacji zaawansowanej	

Pełny przykład ze wszystkim:

```
UTWÓRZ TABELĘ zamówienia (
        id                      BIGINT AUTO_INCREMENT KLUCZ PODSTAWOWY,
        identyfikator_klienta   BIGINT NOT NULL,
        całkowity               DECIMAL(12,2) NOT NULL DOMYŚLNIE 0,00,
        status                  ENUM('oczekujące', 'zapłacone', 'wysłane') NIE NULL DOMYŚLNIE
„oczekujący” ,
        data_zamówienia         DATA/CZAS DOMYŚLNY ZNACZNIK CZASU BIEŻĄCY,

```

Machine Translated by Google

```
        KLUCZ OBCY (client_id) REFERENCJE clients(id)
            OGRANICZ USUWANIE AKTUALIZACJI KASKADOWYCH,

        INDEX client_idx_state (client_id, stan)                    -- indeks złożony
);
```

Indeksy – sekret szybkości

Indeks jest podobny do indeksu książki: umożliwia znalezienie strony bez konieczności czytania całej książki.

SQL — Utwórz indeks po

```
UTWÓRZ INDEKS date_idx ON orders(order_date);
```

Unikalny indeks złożony (bardzo wydajny)

```
UTWÓRZ UNIKALNY INDEKS customer_idx_date NA orders(customer_id, order_date);
```

Pełnotekstowy indeks (dla wyszukiwań typu LIKE '%text%')

```
UTWÓRZ INDEKS PEŁNOTEKSTOWY idx_descripcion ONproductos (descripcion);
```

> Zaindeksuj wszystkie elementy, które zamierzasz filtrować, w klauzuli WHERE lub uporządkuj je za pomocą ORDER BY.

Każdy indeks zajmuje miejsce na dysku i spowalnia operacje INSERT/UPDATE. Równowaga jest kluczowa.

Machine Translated by Google

6 SQL - Język manipulacji danymi (DML).

Narzędzia DML umożliwiają dodawanie nowych pól, zmienianie zawartości istniejących pól, usuwanie pól i wykonywanie tych wszystkich czynności w sposób bezpieczny, zapobiegający zagubieniu lub uszkodzeniu danych.

DML (Data Manipulation Language) jest wyłącznie odpowiedzialny za pracę z Dane, które już istnieją w tabelach. Nie zmieniają struktury (to DDL).

Dzisiaj opanujemy każde polecenie krok po kroku, wraz z jego dokładną składnią i przykładami. przypadki z życia wzięte, typowe błędy i najlepsze praktyki.

1. Wprowadzanie danych: INSERT INTO

Tego polecenia używamy, aby dodać nowe rekordy (wiersze) do tabeli.

Podstawowa składnia

```
INSERT INTO nazwa_tabeli (kolumna1, kolumna2, ..., kolumnaN)
WARTOŚCI (wartość1, wartość2, ..., wartośćN);
```

Wersja bez określania kolumn (tylko gdy wstawiamy WSZYSTKIE kolumny i w dokładna kolejność tabeli):

```
INSERT INTO table_name VALUES (wartość1, wartość2, ..., wartośćN);
```

Wstawianie wielu wierszy na raz (bardzo wydajne):

```
WSTAW DO produktów (nazwa, cena, stan magazynowy)
WARTOŚCI
        („Laptop Dell”, 899,99, 15),
        (Mysz bezprzewodowa, 29,50, 80),
        ('Klawiatura mechaniczna', 75,00, 40);
```

Machine Translated by Google

Kompletny przykład praktyczny

Załóżmy, że mamy taką tabelę:

```
1 CREATE TABLE klienci (
2         id INT AUTO_INCREMENT KLUCZA PODSTAWOWEGO ,
3         nazwa VARCHAR(100) NOT NULL,
4         e-mail VARCHAR(100) UNIQUE,
5         data_rekordu DATA DOMYŚLNA DATA_BIEŻĄCA
6 );
```

Dodaliśmy nowego klienta:

```
1 WSTAW DO Clientes (nombre, e-mail) WARTOŚCI („María López”, „maria.lopez@ejemplo.com”);
```

Najlepsze praktyki i wskazówki

- Zawsze zapisuj listę kolumn. Dzięki temu, jeśli później dodasz kolumnę, łatwo znajdziesz potrzebne kolumny. Nowy, Twój INSERT się nie złamie.
- Użyj opcji DEFAULT, jeśli kolumna na to pozwala i nie chcesz wprowadzać wartości.
- Aby wstawić dane z innej tabeli: INSERT INTO destination_table SELECT ... Z tabeli źródłowej;
- Częsty błąd: zapominanie o cudzysłowie w tekście lub wstawianie przecinków w niewłaściwym miejscu.

Mini ćwiczenie do ćwiczeń

Spróbuj dodać 3 kolejnych klientów.

Następnie zweryfikuj za pomocą SELECT * FROM clients;

2. Aktualizacja rekordu: AKTUALIZACJA

To polecenie modyfikuje istniejące dane. Uwaga! Bez klauzuli WHERE nastąpiłaby aktualizacja.

wszystkie rekordy w tabeli.

Machine Translated by Google

Składnia

```
1 AKTUALIZACJA nazwa_tabeli
2 SET column1 = new_value1,
3          kolumna2 = nowa_wartość2,
4          ...
5. Warunek WHERE ;
```

Przykłady ze świata rzeczywistego

```
1 SQL — Zwiększ cenę pojedynczego produktu
2 produkty UPDATE
Cena zestawu 3 = cena *          1.10
4 GDZIE id = 5;
```

-- Aktualizuj wiele pól jednocześnie

```
1 AKTUALIZACJA klientów
2 USTAW e-mail = 'nowy.email@mail.com',
3          data_rejestracji = '2026-02-18'
4 GDZIE imię = „María López”;
```

-- Aktualizacja za pomocą podzapytania

```
1. AKTUALIZUJ produkty
2 SET stock = stock - 1
3 GDZIE id = (WYBIERZ product_id Z sprzedaży GDZIE id = 342);
```

Rada Bezpieczeństwa

Zawsze najpierw napisz polecenie SELECT z tym samym warunkiem, aby zobaczyć, które wiersze zostaną wybrane.

oddziaływać:

```
1 WYBIERZ *  Z produktów WHERE id = 5;          -- sprawdź przed
```

następnie zmieniają SELECT na UPDATE

Machine Translated by Google

3. Usuwanie danych: USUŃ

Usuwa całe rekordy. Tak jak UPDATE, bez WHERE usuwa WSZYSTKO.

Składnia

```
USUŃ Z nazwa_tabeli
warunek WHERE ;
```

Przykłady

```
SQL — Usuń określonego klienta
USUŃ Z KLIENTÓW
GDZIE id = 7;
```

-- Usuń wszystkie produkty niedostępne w magazynie

```
USUŃ Z produktów
GDZIE zapas <= 0;
```

-- Usuń za pomocą JOIN (MySQL/MariaDB)

```
USUŃ p Z produktów p
WEWNĘTRZNE POŁĄCZENIE kategorii c WŁĄCZONE p.categoria_id = c.id
GDZIE c.nombre = 'Descatalogados';
```

Ważny

- Zawsze, zawsze, zawsze używaj WHERE w poleceniach DELETE i UPDATE
- Polecenie DELETE jest wolniejsze niż TRUNCATE, ponieważ aktywuje wyzwalacze i Rejestruje każdy usunięty wiersz w dzienniku transakcji.
- Nie można użyć polecenia DELETE do usunięcia tylko niektórych kolumn (w tym celu należy użyć polecenia UPDATE SET column = NULL).

Machine Translated by Google

4. Transakcje: BEGIN, COMMIT, ROLLBACK

Transakcja to grupa operacji SQL wykonywanych jako pojedyncza jednostka.

Albo wszystko idzie dobrze, albo nic się nie zmienia.

Główne polecenia

BEGIN; lub START TRANSAKCJI; rozpoczyna transakcję

COMMIT; trwale zapisuje wszystkie zmiany

ROLLBACK; cofa wszystkie zmiany od BEGIN

Klasyczny przykład: przelew bankowy

```
POCZĄTEK;

AKTUALIZUJ konta
SET saldo = saldo - 500,00
GDZIE id = 101; -- konto źródłowe

AKTUALIZUJ konta
SET saldo = saldo + 500,00
GDZIE id = 205; -- konto docelowe

COMMIT; -- wszystko gra!
```

Jeśli coś pójdzie nie tak (np. konto 205 nie istnieje),

```
COFNIJ;
```

Większość klientów (MySQL Workbench, DBeaver itp.) może również używać

Przycisk „Wycofaj” lub „Zatwierdź”.

Tryb automatycznego zatwierdzania

Domyślnie każde polecenie jest transakcją automatyczną. Aby ją wyłączyć

Tymczasowo używamy BEGIN.

Machine Translated by Google

5. Właściwości ACID – gwarancja bezpieczeństwa

Każda transakcja w relacyjnych bazach danych musi być zgodna z 4 właściwościami ACID.

Spróbujmy zrozumieć je tak, jakbyśmy byli bankierami:

Nieruchomość	Oznaczający	Prawdziwy przykład	Co się stanie, jeśli tego nie zrobię? spełnia
Atomowość (Atomowość)	„Wszystko albo nic”	Przelew pieniędzy	Pieniądze są odejmowane, ale nie dodaje
Konsystencja (Konsystencja)	Baza danych pochodzi z jednego stanu ważne w innym ważnym stanie.	Saldo nigdy nie będzie dać wynik negatywny	Saldo pozostało na poziomie -200
Izolacja (Izolacja)	Transakcje nie kolidują ze sobą.	Dwie osoby kupują ostatni bilet	Sprzedaje się dwa bilety, mimo że dostępny jest tylko jeden.
Trwałość (Trwałość)	Po ZATWIERDZENIU zmiany zostaną zapisane nawet po wyłączeniu serwera.	Potwierdzenie zakupu w sklepie internetowym	Zakup przepada po dokonaniu płatności

Poziomy izolacji

ODCZYTAJ NIEZAKOMITOWANE (możesz odczytać dane, które później zostaną odwrócone „brudny odczyt”)

ODCZYT ZATWIERDZONY (najczęściej spotykany)

POWTARZALNY ODCZYT (domyślne ustawienie MySQL)

SERIALIZOWALNY (najbezpieczniejszy i najwolniejszy)

Pamiętać

INSERT wstawia nowe dane

UPDATE zmienia istniejące dane i zawsze używaj WHERE w poleceniu

DELETE usuwa dane i zawsze używaj WHERE w instrukcji

Transakcje pozwalają nam grupować operacje.

Właściwości ACID są powodem, dla którego w przypadku ważnych kwestii (banki, rezerwacje lotnicze, systemy opieki zdrowotnej itp.) polegamy na bazach danych.

Machine Translated by Google

Podstawowe zapytania w MySQL

Kiedy mówimy o podstawowych zapytaniach SQL w MariaDB, wszystko zaczyna się od Polecenie SELECT jest niczym główny silnik wyodrębniający dane z tabel. Wyobraź sobie, że masz bazę danych pełną informacji, np. ogromną bibliotekę, a SELECT jest narzędziem, którego używasz, aby wyszukać dokładnie te książki, których potrzebujesz.

Jego podstawowa składnia wygląda mniej więcej tak:

```
1 WYBIERZ kolumny Z tabeli;
```

Ale możesz to komplikować tak bardzo, jak chcesz.

Na przykład, jeśli chcesz uzyskać wszystkie kolumny z tabeli o nazwie „pracownicy", wykonaj następujące polecenie:

```
1 WYBIERZ * Z pracowników;
```

gdzie gwiazdka oznacza „wszystkie".

Jeśli interesują Cię tylko nazwiska i pensje, to

```
1 WYBIERZ nazwę, pensję OD pracowników;
```

Ogólna struktura obejmuje polecenie SELECT w celu wybrania danych oraz polecenie FROM w celu określenia źródła. Przychodzą, a następnie możesz dodać więcej klauzul, takich jak GDZIE filtrować, ORDER BY zamówić, itd.

W MariaDB działa to szybko, ponieważ optymalizuje zapytania, ale zawsze warto określić tylko te kolumny, których potrzebujesz, aby uniknąć przeciążenia systemu.

Rozważmy przykład ze świata rzeczywistego: załóżmy, że mamy tabelę „produkty" z identyfikatorem, nazwą i ceną. Proste zapytanie może wyglądać następująco: SELECT name, price FROM products; zwróci ono listę nazw z cenami, tak jakbyś przeglądał katalog.

Ale pójdźmy dalej, ponieważ SELECT nie jest przeznaczony tylko do prostych tabel; można go używać z Połączenia umożliwiające łączenie danych z wielu tabel, choć zostawimy to dla modułów bardziej zaawansowane, ale powiem ci teraz, że składnia rozszerza się do

```
1 SELECT kolumna1, kolumna2 FROM tabela1 JOIN tabela2 ON warunek;
```

Machine Translated by Google

W MariaDB jedną z zalet jest obsługa aliasów dla kolumn i tabel, takich jak

```
1 WYBIERZ nazwę JAKO 'Imię i nazwisko pracownika' Z pracownicy JAKo prac;
```

Dzięki temu wyniki stają się bardziej czytelne, zwłaszcza w raportach lub podczas Eksportujesz dane do narzędzi takich jak Excel.

Inny praktyczny szczegół: jeśli Twoja baza danych zawiera miliony wierszy, nieograniczona liczba instrukcji SELECT może być powolna, dlatego MariaDB pozwala na dodanie na końcu instrukcji LIMIT w następujący sposób:

```
1 WYBIERZ * Z dzienników LIMIT 100;
```

aby pobrać tylko pierwsze 100 wierszy, idealne do testowania lub stronicowania w aplikacjach sieć.

Rozważmy codzienny scenariusz: wyobraźmy sobie sklep internetowy z tabelą „zamówienia", która Zawiera identyfikator zamówienia, identyfikator klienta, datę i sumę całkowitą. Podstawowe zapytanie wyglądałoby następująco:

```
1 SELECT order_id, data, suma Z zamówień;
```

który umożliwia szybkie podsumowanie ostatnich sprzedaży.

Jeżeli chcesz coś obliczyć w danej chwili, możesz użyć wyrażeń w poleceniu SELECT, takich jak:

```
1 WYBIERZ nazwę, cenę * 1.21 AS cena_z_podatkiem OD produktów;
```

stosowanie podatku bezpośrednio w zapytaniu, bez konieczności modyfikowania tabeli oryginalny.

Pokazuje to, że SELECT to nie tylko ekstrakcja, ale także lekka transformacja danych. W środowiskach produkcyjnych z MariaDB należy pamiętać, że zapytania SELECT są tylko do odczytu i niczego nie modyfikują, co czyni je bezpiecznymi do celów eksperymentalnych. Należy jednak zawsze sprawdzać uprawnienia użytkownika, aby zapobiec nieautoryzowanemu dostępowi.

Przejdźmy teraz do filtrowania danych za pomocą klauzuli WHERE, która jest niezbędna, aby uniknąć Wprowadź wszystkie niepotrzebne informacje. Polecenie WHERE pojawia się po słowie FROM i pozwala ustawić warunki, aby wybrać tylko wiersze spełniające określone kryteria.

Na przykład, jeśli chcesz zatrudnić tylko pracowników z pensjami powyżej 2000, powinieneś:

```
1 WYBIERZ * Z pracownicy GDZIE pensja > 2000;
```

Machine Translated by Google

To jak nakładanie filtra na wyszukiwarkę Google. Można go łączyć z innymi.
elementów, a w MariaDB obsługuje złożone wyrażenia.

Ważnym szczegółem jest to, że WHERE ocenia każdy wiersz indywidualnie, więc jeśli Twoja tabela
To duża sprawa, należy myśleć o przyspieszeniu w kategoriach indeksów.

Przykład praktyczny: w tabeli „sprzedaż" zawierającej datę, produkt i ilość, aby zobaczyć sprzedaż
ponad 10 jednostek w styczniu 2023 r.,

```
1 WYBIERZ produkt, ilość ZE sprzedaży GDZIE ilość > 10 I data JAK
'2023-01%';
```

Tutaj już trochę napomknąłem o operatorach, ale jak widać WHERE sprawia, że zapytanie jest precyzyjne i użyteczne.

Przyjrzyjmy się temu bliżej:
W MariaDB polecenie WHERE może obejmować podzapytania, takie jak

```
1 WYBIERZ * Z produktów GDZIE cena > (WYBIERZ ŚREDNIA(cena) Z
produkty);
```

Pozwala to na odfiltrowanie produktów o ponadprzeciętnych parametrach, co przydaje się przy analizie cen.

Innym aspektem jest obsługa wartości null; jeśli użyjesz WHERE column = NULL, nie zadziała.
Ponieważ NULL nie jest równy niczemu, nawet sobie samemu, należy używać IS NULL lub IS NOT
NULL. Na przykład,

```
1 WYBIERZ * Z klientów , GDZIE telefon JEST NULL;
```

do znajdowania klientów bez zarejestrowanego numeru, przydatne w kampaniach marketingowych.

W scenariuszach z życia wziętych, takich jak baza danych szpitala zawierająca dane pacjentów, wiek i objawy
i entry_date, możesz to zrobić

```
1 WYBIERZ imię, wiek Z pacjentów GDZIE objawy JAK '%gorączka%' I wiek
< 18;
```

w celu identyfikacji dzieci z gorączką i priorytetowego zapewnienia im opieki.

Machine Translated by Google

MariaDB optymalizuje klauzule WHERE z indeksami złożonymi, jeśli je utworzysz, na przykład CREATE INDEKS idx_ventas ON ventas (producto, cantidad); co przyspiesza częste zapytania.

Nie zapomnij, że WHERE stosuje się przed innymi klauzulami, takimi jak GROUP BY, więc Aby zwiększyć wydajność, filtruj wcześnie.

Inny rozszerzony przykład: załóżmy, że mamy tabelę „książki" z tytułem, autorem, rokiem wydania i gatunkiem. W przypadku książek science fiction wydanych po 2000 roku,

```
1 WYBIERZ tytuł, autora Z książek GDZIE gatunek = 'Science fiction' I rok_publikacji >
2000; proste, ale jeśli dodasz OR , aby uwzględnić fantasy, GDZIE gatunek = 'Sci-Fi' LUB
gatunek = 'Fantasy' ORAZ rok_wydania > 2000;
```

Należy zachować ostrożność w kolejności, ponieważ AND ma pierwszeństwo, dlatego należy używać nawiasów:

GDZIE (gatunek = 'Science fiction' LUB gatunek = 'Fantasy') ORAZ rok_publikacji > 2000;

Pozwala to uniknąć typowych błędów i gwarantuje poprawne wyniki.

Następnie klauzula ORDER BY umożliwia uporządkowanie

Wyniki, które mają sens wizualny lub logiczny. Są one przedstawiane na końcu konsultacji i określane przez Ciebie. kolumna, według której należy sortować, a następnie ASC, aby sortować rosnąco (od najmniejszej do największej), co (domyślnie) lub DESC (w kolejności malejącej).

Na przykład,

```
1 WYBIERZ nazwę, wynagrodzenie OD pracowników KOLEJNOŚĆ WEDŁUG wynagrodzenia OPIS;
```

Wyświetla listę pracowników od najwyższego do najniższego wynagrodzenia, co jest przydatne przy tworzeniu rankingów.

Można sortować według wielu kolumn:

PORZĄDEK WEDŁUG DZIAŁÓW (ROŚNIEJ), PIENIĘDZY (MNIEJ NIŻ DZIAŁ); najpierw pogrupuj alfabetycznie według działów, a w obrębie każdego działu – pensje od najwyższych do najniższych.

W MariaDB, jeśli występują wartości null, ASC umieszcza je na początku, a DESC na końcu, co jest Subtelny, ale ważny szczegół w przypadku niekompletnych danych. Wypróbuj to na tabeli „klienci". z imieniem i wiekiem:

```
1 WYBIERZ * OD klientów KOLEJNOŚĆ WEDŁUG wiekuASC ;
```

Najpierw zobaczysz tych najmłodszych, jakbyś układał listę gości.

według wieku.

Machine Translated by Google

Rozwińmy: ORDER BY można stosować z wyrażeniami takimi jak ORDER BY DŁUGOŚĆ(nazwa) ASC; sortowanie według długości nazwy, przydatne w analizie danych tekstowy.

W MariaDB obsługuje sortowanie uwzględniające akcenty lub wielkość liter, np. Na przykład, jeśli w tabeli użyto utf8mb4_general_ci, kolejność jest ustalana bez względu na wielkość liter, ale w zapytaniu można określić klauzulę COLLATE: ORDER BY name COLLATE utf8mb4_bin; uwzględnia wielkość liter.

W przykładzie handlu elektronicznego z tabelą „produkty" obejmującą stany magazynowe i sprzedaż,

```
1 WYBIERZ nazwę, stan magazynowy OD produktów ZAMÓW WEDŁUG stanu magazynowego OPIS LIMIT 10;
```

Wyświetla 10 produktów o największej liczbie zapasów, co ułatwia zarządzanie zapasami.

Jeżeli połączysz z WHERE, WYBIERZ * Z zamówień WHERE data > '2023-01-01' ORDER BY suma DESC; sortuje ostatnie zamówienia według kwoty malejącej, idealne dla zidentyfikować głównych klientów.

Inna sztuczka: aby posortować listę według pozycji, użyj FIELD, np. ORDER BY. POLE(status, „Oczekujące", „Wysłane", „Dostarczone"); które sortuje według kolejności spersonalizowane, niealfabetyczne.

W dużych bazach danych polecenie ORDER BY bez indeksu może być wolne, ponieważ wymaga Sortowanie w pamięci; MariaDB korzysta w razie potrzeby z tabel tymczasowych, więc indeksuje kolumny używane w ORDER BY. Wyobraź sobie tabelę „wydarzenia" z datą, lokalizacją i... uczestnicy:

```
1 WYBIERZ miejsce, uczestników Z wydarzeń KOLEJNOŚĆ WEDŁUG uczestników OPISUJĄCO, data ASC;
```

Najpierw według największej liczby uczestników, a w przypadku remisu — według najwcześniejszej daty, co jest idealne do raportów popularności.

Jeśli chodzi o operatory porównawcze i logiczne, to są to bloki konstrukcja umożliwiająca stworzenie warunków w WHERE lub gdzie indziej.

Symbole porównania obejmują = dla równości, <> lub != dla nierówności, >, <, >=, <= dla starszy lub młodszy.

Machine Translated by Google

Następnie operatory logiczne: AND, aby oba warunki były prawdziwe, OR, aby gdy
Chyba że jedno jest prawdziwe, a NIE neguje. Na przykład,
WYBIERZ * Z produktów GDZIE cena > 50 I stan magazynowy < 10;
Oferuje drogie produkty z niskim stanem magazynowym. IN świetnie sprawdza się w przypadku list: WHERE id IN (1, 3, 5);
jakby powiedzieć „jeden z tych”.

BETWEEN dla zakresów: WHERE data BETWEEN '2023-01-01' AND '2023-12-31'; obejmuje punkty końcowe. Oraz
LIKE dla wzorców, z % jako symbolem wieloznacznym: WHERE nazwa LIKE
'A%'; dla nazw zaczynających się na literę A.

Inni operatorzy

W MariaDB funkcja LIKE domyślnie nie uwzględnia wielkości liter w standardowym sortowaniu, ale można to zrobić
Dostosuj to. Przykład łączony: SELECT * FROM users WHERE age BETWEEN
18 I 35 LUB (miasto = „Madryt” I NIE aktywne); filtruje osoby młode lub pochodzące z Madrytu
nieaktywny, pokazuje jak zagnieżdżać za pomocą nawiasów dla przejrzystości.
Przyjrzyjmy się bliżej każdemu z nich: operator = jest ścisły dla typów, ale MariaDB tak robi
niejawne konwersje, takie jak porównywanie ciągu znaków z liczbą całkowitą, jeśli jest to możliwe.
W przypadku nierówności <> to standardowy zapis SQL, ale != to powszechny alias.

W przypadku bramek logicznych operator AND oblicza wynik od lewej do prawej i powoduje zwarcie, jeśli pierwszy wynik jest fałszywy.
które optymalizuje. LUB podobnie, jeśli pierwsze jest prawdziwe, drugie nie jest oceniane. NIE odwraca, przydatne.
z IN: NOT IN (wartości); do wykluczenia. BETWEEN jest inkluzywne i działa z datami.
liczby lub ciągi znaków.

W przypadku dat należy upewnić się, że używasz formatu ISO. LIKE obsługuje _ dla pojedynczych znaków: LIKE 'A__'; dla
Trzy litery zaczynające się na A. W przypadku wyrażeń regularnych MariaDB ma REGEXP dla zaawansowanych wzorców:
WHERE text REGEXP '^A.*b$'; dla zaczyna się od A, kończy na b. Przykład w tabeli.
„e-maile”:
WYBIERZ * Z użytkowników, GDZIE adres e-mail TAK JAK '%@gmail.com' I wiek > 25 LUB (kraj = 'Hiszpania' I NIE
Zweryfikowany); łączy się w celu kierowania
kampaniami.

Inny:
GDZIE wynik pomiędzy 80 a 100 i kategoria W ('A', 'B');

Machine Translated by Google

Aby uzyskać wysokie wyniki w określonych kategoriach. W przypadku wydajności, unikaj LIKE z % na początku, jeśli Możliwe, ponieważ nie używa w pełni indeksowanych indeksów; w niektórych przypadkach ILIKE jest lepszy, ale MariaDB używa JAK nieczuły.

Test z prawdziwymi danymi: w „inwentarzu”,

WYBIERZ pozycję Z magazynu GDZIE ilość < 5 LUB (cena > 100 I NIE

dostępne);

identyfikuje tanie lub drogie przedmioty, które są niedostępne.

Przechodząc do funkcji agregujących, są one jak podsumowania matematyczne, które stosuje się do grup danych. COUNT() zlicza wiersze:

SELECT COUNT(*) FROM orders; informuje, ile

jest zamówień. SUM()

suma wartości:

WYBIERZ SUMA(cena) ZE sprzedaży;

dla całkowitej sprzedaży.

Średnie AVG():

WYBIERZ ŚREDNIĄ(pensję) Z pracowników;

za przeciętną pensję.

Funkcje MAX() i MIN() podają wartość maksymalną i minimalną:

WYBIERZ MAX(wiek) Z klientów;

dla najstarszego.

Nie zwracają pojedynczych wierszy, lecz pojedyncze wartości lub grupy wartości.

W MariaDB można ich używać razem z DISTINCT w celu zliczania unikatowych

elementów: COUNT(DISTINCT user_id).

Przykład w tabeli „egzaminy” z ocenami: S

WYBIERZ AVG(uwaga) JAKO średnią, MAX(uwaga) JAKO maksimum Z egzaminów GDZIE

temat = 'Matematyka';

i aliasy, takie jak AS, do nadawania nazw wynikom, dzięki czemu dane wyjściowe stają się bardziej czytelne.

Agregacje

Machine Translated by Google

Przyjrzyjmy się bliżej: agregacje obsługują wartości null, ignorując je, z wyjątkiem COUNT(*), które Liczy wszystko. Dla ciągów znaków jest CONCAT, ale dla agregatów jest GROUP_CONCAT: SELECT GROUP_CONCAT(name) FROM employees; łączy rozdzielone nazwiska przecinkiem. W finansach SUMA(suma) dla sald.

MariaDB obsługuje funkcje okien w najnowszych wersjach, ale w przypadku podstaw należy trzymać się Czy.

Rozszerzony przykład: w „transakcjach" z kwotą i typem,

WYBIERZ SUMA(kwota) JAKO dochód Z transakcji GDZIE typ = 'Dochód';

przeciwko

WYBIERZ ŚREDNIĄ(kwota) Z transakcji GDZIE typ = 'Wydatek';

do analizy budżetu.

Połącz z GDZIE:

WYBIERZ MIN(cena) Z produktów GDZIE kategoria = 'Elektronika';

najniższa cena w tej kategorii.

Aby zapewnić dokładność, AVG używa wartości dziesiętnych; MariaDB używa wartości zmiennoprzecinkowych, ale można użyć CAST do DZIESIĘTNY. Inne:

WYBIERZ LICZBA(*) JAKO suma, LICZBA(ODMIENNE miasto) JAKO unikatowe_miasta Z klienci;

całkowita liczba klientów i unikalne miasta, przydatne do analizy demograficznej.

Na koniec grupowanie za pomocą GROUP BY i HAVING przenosi agregacje na inny poziom poziom, umożliwiając podsumowania według kategorii. GROUP BY następuje po WHERE i Grupuj wiersze według kolumny:

WYBIERZ dział, LICZ(*) Z pracowników GRUPUJ WEDŁUG działu;

Policz pracowników w każdym dziale.

HAVING filtruje te grupy podobnie jak WHERE, ale dla agregatów:

MAJĄC LICZBĘ(*) > 5;

tylko działy zatrudniające powyżej 5 pracowników.

Nie można używać polecenia WHERE do bezpośredniego filtrowania agregatów; tu właśnie pojawia się polecenie HAVING. MariaDB, kolejność jest następująca: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.

Przykład: w „sprzedaży" z produktem i ilością,

Machine Translated by Google

WYBIERZ produkt, SUMA(ilość) JAK całkowita_sprzedaż ZE sprzedaży GRUPUJ WEDŁUG produktu
MAJĄC total_sold > 100 ZAMÓW WEDŁUG total_sold DESC;
Wyświetla najlepiej sprzedające się produkty, posortowane od najchętniej do najgorzej sprzedających się, co jest doskonałym rozwiązaniem do raportów.
spis.

Rozszerzenie: GROUP BY może obejmować wiele kolumn: GROUP BY rok, miesiąc; dla
miesięczne sumy w ciągu roku.

MariaDB umożliwia ROLLUP dla sum częściowych: GROUP WEDŁUG roku Z ROLLUP; dodaje wiersz
całkowity.
HAVING używa agregatów: HAVING AVG(pensja) > 3000;

Przykład w „projektach”:
WYBIERZ zespół, MAX(budżet) JAK max_budżet Z projektów GRUPUJ WEDŁUG zespołu
MAJĄC max_pres > 10000;
zespoły realizujące kosztowne projekty.

Połącz wszystko: WYBIERZ kategorię, LICZ(*) JAK num, SUMA(sprzedaż) JAKo suma OD
pozycje GDZIE data > '2022' GRUPUJ WEDŁUG kategorii MAJĄC liczbę > 10 ZAMÓW WEDŁUG sumy
OPIS;
filtr, grupa, grupy filtrów, sortowanie.

Pod względem wydajności, GROUP BY korzysta z indeksów, jeśli są dostępne; w przeciwnym razie stosuje sortowanie. W przypadku dużych zbiorów danych,
Rozważ partycje w MariaDB. Inny scenariusz: tabela „logs” z użytkownikiem, akcją i datą;
WYBIERZ użytkownika, LICZ (akcja) JAKo aktywności Z dzienników GRUPUJ WEDŁUG użytkownika
Posiadanie aktywności > 50; aktywni użytkownicy.

Machine Translated by Google

Moduł 8 – Zaawansowane zapytania MySQL/MariaDB

Zaawansowane zapytania SQL zmieniają sposób, w jaki wchodzimy w interakcję z przechowywanymi informacjami. Techniki te nie tylko umożliwiają dokładne wyszukiwanie danych, ale także ułatwiają dogłębną analizę, która prowadzi do podejmowania świadomych decyzji biznesowych.

W tym module zajmiemy się połączeniami umożliwiającymi łączenie tabel, podzapytaniami dla zapytań w obrębie zapytań, operatorami zbiorów umożliwiającymi manipulowanie wynikami tak, jakby były zbiorami matematycznymi, wykorzystaniem aliasów w celu poprawy czytelności oraz zagnieżdżonymi i skorelowanymi zapytaniami, które dodają warstwy inteligentnej złożoności.

Zilustrujemy to wszystko praktycznymi przykładami opartymi na scenariuszu zarządzania zasobami ludzkimi w firmie, gdzie mamy tabele pracowników i działów. W ten sposób każda koncepcja będzie oparta na rzeczywistych sytuacjach, które można odtworzyć we własnym środowisku programistycznym.

Aby umieścić nasze przykłady w szerszym kontekście, rozważmy typową bazę danych zawierającą tabelę „pracownicy", która rejestruje takie dane, jak unikalny identyfikator każdego pracownika, imię i nazwisko, dział oraz wynagrodzenie. Tabela ta może zawierać rekordy pracowników, którzy nie są przypisani do żadnego działu, co odzwierciedla typowe sytuacje w organizacjach przechodzących transformację.

Z drugiej strony, tabela „departments" przechowuje identyfikator, nazwę i lokalizację każdego działu. Dodatkowo, aby zbadać operatory zbiorów, wprowadzimy tabelę „formal_employees", która ma podobną strukturę i zawiera zarówno rekordy współdzielone, jak i unikatowe. Dane te pozwolą nam zobaczyć w praktyce, jak zaawansowane zapytania dostarczają cennych informacji, takich jak identyfikacja osób o zarobkach powyżej średniej czy łączenie danych historycznych bez duplikatów.

Zacznijmy od omówienia złączeń, czyli unii tabel, które stanowią istotę wielu złożonych zapytań, umożliwiając powiązanie informacji rozproszonych w wielu encjach. Proces łączenia opiera się na porównywaniu wartości między kolumnami różnych tabel zgodnie z określonym warunkiem, generując przefiltrowany iloczyn kartezjański, który zachowuje tylko odpowiednie kombinacje.

Bez połączeń bylibyśmy ograniczeni do pracy z izolowanymi danymi, ale dzięki nim możemy budować zintegrowane widoki, które symulują pojedynczy, spójny zestaw informacji. Typ
Najczęściej używanym jest INNER JOIN, który działa jak ścisły filtr, zwracając tylko wiersze, w których występuje dokładne dopasowanie między tabelami. To idealne rozwiązanie.

Gdy chcemy skupić się wyłącznie na powiązanych danych, na przykład wyświetlając listę tylko tych pracowników, którzy faktycznie należą do aktywnego działu, składnia wymaga określenia tabel, warunku łączenia z opcją ON oraz kolumn do wybrania. Rozważmy zapytanie, które pobiera imię i nazwisko pracownika oraz nazwę działu:

Machine Translated by Google

```
WYBIERZ e.nombre, e.apellido, d.nombre_departamento
OD pracowników i
INNER JOIN działy d ON e.department_id = d.department_id
```

W tym przypadku aliasy `eyd` upraszczają odwołanie do każdej tabeli, unikając niejednoznaczności w przypadku, gdy kolumny mają tę samą nazwę. Po uruchomieniu tego zapytania na naszych przykładowych danych, wyniki byłyby dokładnie takie: Juan Pérez z działu sprzedaży, María Gómez z działu sprzedaży, Pedro López z działu IT, Luis Rodríguez z działu HR i Elena Sánchez z działu IT.
Jak widać, pojawiają się tylko pracownicy, którym przypisano odpowiedni dział, co automatycznie eliminuje wszelkie niekompletne rekordy.

To zachowanie sprawia, że INNER JOIN idealnie nadaje się do raportów wewnętrznych, w których Dokładność jest kluczowa, jak w przypadku pulpitu nawigacyjnego produktywności dla poszczególnych działów. Jednak częstym błędem jest zapominanie, że jeśli istnieje wiele dopasowań, wynik może być zawyżony; na przykład, jeśli pracownik jest przypisany do kilku działów, zobaczymy duplikaty, chyba że dodamy DISTINCT lub dostosujemy logikę biznesową.

Co więcej, z punktu widzenia wydajności, kluczowe jest tworzenie indeksów w kolumnach Identyfikatory działów z obu tabel są używane, ponieważ silnik bazy danych wykorzysta je do przyspieszenia wyszukiwania zamiast skanowania wszystkich informacji. Na przykład w rzeczywistych projektach e-commerce ten sam schemat jest stosowany do dopasowywania klientów do zamówień i wyświetlania tylko zakończonych transakcji, zapobiegając w ten sposób przeładowaniu raportu niepotrzebnymi informacjami.

Przechodząc do LEFT JOIN, ten wariant zapewnia, że wszystkie wiersze w tabeli po lewej stronie zostaną uwzględnione w wyniku, niezależnie od tego, czy dopasowanie zostanie znalezione w tabeli po prawej stronie. W przypadku braku dopasowania, kolumny w tabeli po prawej stronie są wypełniane wartościami NULL, co otwiera drogę do analizy brakujących lub niekompletnych danych.

Wyobraź sobie, że musisz zidentyfikować pracowników bez przypisanego działu na potrzeby audytu kadrowego; klauzula LEFT JOIN pozwala Ci to łatwo zrobić, dodając kolejny warunek, taki jak WHERE d.department_name IS NULL. Zapytanie bazowe wyglądałoby następująco:

```
WYBIERZ e.nombre, e.apellido, d.nombre_departamento
OD pracowników i
LEWY DOŁĄCZ do działów d WŁĄCZONY e.department_id = d.department_id
```

Po uruchomieniu zobaczylibyśmy Juana Péreza i Maríę Gómez w dziale sprzedaży, Pedro Lópeza w dziale IT, Anę Martínez z NULL, Luisa Rodrígueza w dziale HR, Elenę Sánchez z IT i Carlosa Fernándeza z NULL. Zwróć uwagę, jak Ana i Carlos pojawiają się z NULL w dziale, podświetlając
w swojej szczególnej sytuacji. Ten typ łączenia jest nieoceniony w scenariuszach migracji danych.

Machine Translated by Google

lub raportów zgodności, w których nie chcesz utracić podstawowych informacji, nawet jeśli nie ma relacji.

Istotną kwestią jest obsługa wartości NULL w kolejnych filtrach, ponieważ porównania takie jak =NULL nie działają; zawsze używaj IS NULL lub IS NOT NULL. Pod względem optymalizacji, chociaż LEFT JOIN może być nieco bardziej kosztowny niż INNER, ponieważ przetwarza więcej wierszy, dobra konstrukcja indeksu znacznie to minimalizuje.

Można to porównać do systemu zarządzania zapasami: łączenie produktów z kategoriami za pomocą LEFT JOIN umożliwia dodanie wszystkich produktów, nawet tych, którym nie przypisano kategorii, co pozwala skorygować wszelkie nieścisłości przed uruchomieniem kampanii sprzedażowej.

RIGHT JOIN działa symetrycznie do LEFT JOIN, ale priorytetyzuje wszystkie wiersze w tabeli po prawej stronie. W naszym przypadku uwzględniałoby to wszystkie działy, nawet jeśli nie mają powiązanych z nimi pracowników, co jest przydatne w raportach z listą pustych lub rozwijających się obszarów. Odpowiednie zapytanie wyglądałoby następująco:

```
WYBIERZ e.nombre, e.apellido, d.nombre_departamento
OD pracowników i
DOŁĄCZ PRAWE DO działów d WŁĄCZONE e.department_id = d.department_id
```

Ponieważ wszystkie działy w naszych danych mają co najmniej jednego pracownika, wynik częściowo pokrywa się z wynikami INNER, ale zasada pozostaje ta sama: Juan Pérez i María Gómez w dziale sprzedaży, Pedro López w dziale IT, Luis Rodríguez w dziale HR i Elena Sánchez
W IT. Gdybyśmy dodali dział bez pracowników, imię i nazwisko pracownika pojawiłyby się z wartością NULL.

Narzędzie to sprawdza się znakomicie w analizie odwrotnej, na przykład przy sprawdzaniu pokrycia działu. W organizacji. Należy pamiętać, że w niektórych systemach, takich jak starsze wersje MySQL, PRAWE DOŁĄCZENIE jest rzadziej używane, ponieważ logikę można odwrócić za pomocą LEFT, ale pozostaje ono standardem w ANSI SQL i jest praktyczne, gdy tabela główna znajduje się po prawej stronie, na przykład w raportach dotyczących lokalizacji fizycznych, które zawsze muszą być uwzględnione.

Aby uzyskać pełne sprzężenie, które obejmuje absolutnie wszystkie rekordy z obu tabel, z wartościami NULL tam, gdzie to konieczne, używamy sprzężenia FULL OUTER JOIN. Nie wszystkie silniki baz danych obsługują je natywnie – na przykład MySQL wymaga symulacji – ale w tych, które je obsługują, takich jak PostgreSQL czy SQL Server, znacznie upraszcza to kod.

Jednym ze sposobów symulacji tego zjawiska jest połączenie LEFT JOIN z RIGHT JOIN, odfiltrowując wartości null po przeciwnej stronie. Używając symulacji w naszym środowisku:

Machine Translated by Google

```
WYBIERZ e.nombre, e.apellido, d.nombre_departamento

OD pracowników i

LEWE DOŁĄCZENIE do działów d WŁĄCZONE e.department_id =
d.id_departamento

UNIA

WYBIERZ e.nombre, e.apellido, d.nombre_departamento

OD pracowników i

DOŁĄCZ PRAWE DO działów d WŁĄCZONE e.department_id =
d.id_departamento

GDZIE e.id_empleado JEST NULL
```

W naszym przykładzie wynik obejmuje Anę Martínez i Carlosa Fernándeza z wartością NULL, a także wszystkich pasujących pracowników w ich odpowiednich działach. Dotyczy to zarówno pracowników bez działu, jak i działów bez pracowników.

FULL OUTER JOIN to potężne narzędzie do przeprowadzania kompletnych uzgodnień w magazynach danych lub integracji starszych systemów, ale ich użycie należy rozważyć w kontekście kosztów obliczeniowych związanych z dużymi wolumenami danych.

Dobrą praktyką jest łączenie go z agregacjami w celu podsumowania rozbieżności i zawsze testowanie na konkretnym silniku projektu, aby uniknąć niespodzianek związanych ze zgodnością.

Teraz, gdy nauczyliśmy się, jak bezpośrednio łączyć tabele, możemy zająć się podzapytaniami, czyli zapytaniami zagnieżdżonymi w innym zapytaniu.
Zapytanie główne. Umożliwia ono wykonywanie obliczeń pośrednich lub dynamiczne filtrowanie bez konieczności tworzenia widoków tymczasowych, co zwiększa elastyczność naszych operacji.
Podzapytanie można umieścić w klauzuli SELECT, aby obliczyć wartości pochodne dla każdego wiersza, w WHERE w przypadku warunków opartych na zagregowanych wynikach lub w FROM, aby działało jako tabela wirtualna.

Zacznijmy od podzapytania w WHERE, które doskonale nadaje się do porównań z dynamicznie obliczanymi średnimi lub sumami. Na przykład, aby znaleźć pracowników, których wynagrodzenie przekracza średnią w całej firmie:

```
WYBIERZ imię, nazwisko, pensję

OD pracowników

GDZIE pensja > (SELECT AVG(pensja) FROM pracownicy)
```

Machine Translated by Google

To wewnętrzne podzapytanie jest wykonywane jako pierwsze i zwraca pojedynczą wartość skalarną, która jest używana w filtrze zewnętrznym. Wyniki w naszych danych to María Gómez z 60000, Luis Rodríguez z 70000 i Elena Sánchez z 65000.

Warto zauważyć, że tylko trzech pracowników spełnia kryteria kwalifikacji, co pokazuje, jak podzapytania upraszczają logikę, która w przeciwnym razie wymagałaby wielu kroków lub zmiennych. W rzeczywistych zastosowaniach, takich jak systemy kadrowo-płacowe, pomaga to szybko identyfikować odchylenia od normy w wynagrodzeniach.

Jeśli jednak podzapytanie jest złożone, może mieć wpływ na wydajność, dlatego zawsze warto rozważyć jego przepisanie jako JOIN z tabelą pochodną, jeśli jest powtarzane wielokrotnie. Co więcej, podzapytania w klauzulach WHERE można łączyć z operatorami takimi jak IN lub EXISTS dla list dynamicznych, co zwiększa ich użyteczność w filtrach wielopoziomowych.

Umieszczając podzapytanie w klauzuli FROM, traktujemy je jako tabelę tymczasową lub pochodną, co pozwala nam na zastosowanie dodatkowych filtrów lub agregacji do wyników pośrednich. Jest to szczególnie przydatne w przypadku zapytań wymagających wcześniejszego grupowania. Załóżmy, że chcemy uzyskać dane z działów zatrudniających więcej niż jednego pracownika:

```
WYBIERZ nazwę_działu, LICZBA_pracowników

FROM (WYBIERZ d.nombre_departamento, COUNT(e.id_empleado) AS
LICZBA_pracowników

        Z działów d

        LEWY DOŁĄCZ pracownicy e WŁ. d.department_id = e.department_id

        GRUPUJ WEDŁUG d.department_name) JAKO sub

GDZIE LICZBA_pracowników > 1
```

Tutaj podzapytanie generuje wirtualną tabelę o nazwie sub, według której przeprowadzamy filtrowanie. Uruchamiając go, otrzymujemy IT z 2 i Sprzedaż z 2. To podejście modularyzuje zapytanie, ułatwiając konserwację w dużych projektach. W kontekście Business Intelligence,
Podzapytania w FROM są kluczowe dla tworzenia warstw agregacji przed ostatecznymi połączeniami.

Pamiętaj, że w tym przypadku podzapytanie musi mieć obowiązkowy alias; w przeciwnym razie silnik bazy danych zgłosi błąd składniowy. Możesz je również rozszerzyć o więcej złączeń wewnętrznych w przypadku bardziej złożonych scenariuszy, takich jak analiza przez...
region i rok.

Dodanie podzapytań do instrukcji SELECT pozwala wzbogacić każdy wiersz wyniku głównego o obliczone informacje, takie jak powiązana wartość z innej tabeli. Na przykład:

Machine Translated by Google

```
SELECT e.nombre, e.apellido,
        (WYBIERZ nombre_departamento Z departamentów d GDZIE d.id_departamento =
e.id_departamento) AS dept
OD pracowników i
```

Działa to jak kolumna pochodna dla każdego pracownika i generuje: Juan Pérez ze Sprzedażą, María Gómez ze Sprzedażą, Pedro López z IT, Ana Martínez z NULL, Luis Rodríguez z HR, Elena Sánchez z IT i Carlos Fernández z NULL.

Choć wygodny w przypadku prostych zapytań, ten typ skalarnego podzapytania może okazać się nieefektywny, gdy jest wykonywany na tysiącach wierszy, gdyż wymaga powtórzenia wyszukiwania. W takich przypadkach LEFT JOIN jest zazwyczaj bardziej optymalny i skalowalny, zwłaszcza w połączeniu z funkcjami okienkowymi w nowoczesnych bazach danych.

Skorelowane podzapytania idą o krok dalej, ponieważ zapytanie wewnętrzne odwołuje się do wartości z zapytania zewnętrznego i jest analizowane wiersz po wierszu, co czyni je użytecznymi w porównaniach zależnych od kontekstu. Klasycznym przykładem jest wyszukiwanie pracowników, którzy zarabiają więcej niż średnia w swoim dziale.

```
WYBIERZ e1.nazwisko, e1.nazwisko, e1.wynagrodzenie
OD pracowników e1
GDZIE e1.pensja > (WYBIERZ ŚREDNIĄ(e2.pensja)
                   OD pracowników e2
                   GDZIE e2.department_id = e1.department_id)
```

Korelacja wykorzystująca e1.id_departamento = e2.id_departamento powoduje przeliczenie podzapytania dla każdego pracownika. Wyniki: María Gómez z 60000 i Elena Sánchez z 65000. Ujawnia to dane według grup bez konieczności stosowania złożonej klauzuli GROUP BY.

W zastosowaniach takich jak analiza wydajności skorelowane kolumny sprawdzają się znakomicie, ale należy zachować ostrożność: w przypadku ogromnych zestawów danych mogą one drastycznie spowolnić wykonywanie kodu, dlatego indeksy w odwoływanych kolumnach są niezbędne. Czasami w nowoczesnych bazach danych lepiej jest je zapisać jako połączenia z funkcjami okna, aby zachować szybkość.

Operatory zbiorów pozwalają nam traktować wyniki całych zapytań jako zbiory matematyczne, łącząc je w precyzyjny sposób. UNION łączy wyniki dwóch lub więcej zapytań, automatycznie usuwając duplikaty i wymagając...

Aby kolumny były zgodne pod względem liczby, typu i kolejności, na przykład, aby uzyskać unikalne nazwiska obecnych i byłych pracowników:

Machine Translated by Google

```
WYBIERZ imię, nazwisko Z pracownicy
UNIA
WYBIERZ imię, nazwisko Z byli_pracownicy
```

Producentami są Ana Martínez, Carlos Fernández, Elena Sánchez, Juan Pérez, Luis Rodríguez, María Gómez, Miguel Torres, Pedro López i Sofía Díaz. Przydatne w przypadku migracji lub skonsolidowanych raportów. Jeśli chcesz zachować duplikaty, użyj UNION ALL, aby uzyskać większą prędkość przy dużych wolumenach.

INTERSECT zwraca tylko wiersze wspólne dla obu zestawów, takie jak pracownicy, którzy występują w obu tabelach: tylko Juan Pérez. Idealne do audytów spójności między systemami starszymi i obecnymi. Na koniec EXCEPT odejmuje drugi zestaw od pierwszego, pokazując, co znajduje się w tabeli „pracownicy", ale czego nie ma w tabeli starszej: Ana Martínez, Carlos Fernández, Elena Sánchez, Luis Rodríguez, María Gómez i Pedro López.

Te operatory wymagają dokładnie takiej samej liczby kolumn i zgodnych typów danych; w przeciwnym razie zakończą się niepowodzeniem. Co więcej, nie zezwalają na użycie instrukcji ORDER BY w każdym poleceniu SELECT, z wyjątkiem końca. W środowiskach wielosystemowych należy zweryfikować konkretne wsparcie, ponieważ niektóre platformy wymagają alternatywnej składni, i zawsze brać pod uwagę wpływ na kolejność wykonywania, aby zachować spójność wyników.

Używanie aliasów ze słowem kluczowym AS jest niezbędne do tymczasowej zmiany nazw tabel i kolumn, poprawy przejrzystości i zapobiegania konfliktom w złożonych zapytaniach. Na przykład, w długich łączeniach, krótkie aliasy, takie jak „e" dla pracowników i „d" dla działów, zwiększają czytelność kodu i umożliwiają jednoznaczne odwoływanie się do kolumn. Ponadto aliasy są obowiązkowe w podzapytaniach pochodnych lub kolumnach obliczeniowych. Rozważ tę ulepszoną wersję:

```
SELECT e.nombre AS NombreEmpleado, d.nombre_departamento AS Departamento
OD pracowników i
INNER JOIN działy d ON e.department_id = d.department_id
```

Wyniki zawierają niestandardowe nagłówki: Juan z działu sprzedaży, María z działu sprzedaży, Pedro z działu IT, Luis z działu HR i Elena z działu IT. Bez aliasów domyślne nazwy kolumn mogłyby być mylące lub identyczne, co utrudniałoby przetwarzanie wyników w aplikacjach. W zapytaniach zagnieżdżonych alias podzapytania pozwala traktować je jak każdą inną tabelę i ułatwia odczytanie kodu przez zespół programistów po miesiącach.

Machine Translated by Google

Wreszcie, zagnieżdżone i skorelowane zapytania integrują wszystkie powyższe elementy, umożliwiając głębokie warstwy logiki w ramach jednego polecenia. Typowe zagnieżdżone zapytanie używa podzapytania w klauzuli FROM z aliasami do późniejszego filtrowania.

```
WYBIERZ podrzędną nazwę pracownika, podrzędny dział
FROM (WYBIERZ e.nombre AS NombreEmpleado, d.nombre_departamento AS
Dział
        OD pracowników i
        INNER JOIN działy d ON e.department_id = d.department_id) AS
zastąpić
GDZIE poddział = 'IT'
```

Zwraca tylko Pedra i Elenę w IT. Skorelowane wartości, jak już widzieliśmy, dodają dynamiki, zależąc od kontekstu zewnętrznego, umożliwiając zaawansowane wzorce, takie jak znajdowanie maksimów w grupie bez użycia komendy GROUP BY.

Łącząc połączenia, podzapytania, operatory i aliasy, możesz tworzyć zapytania, które elegancko i efektywnie rozwiązują złożone problemy biznesowe, zawsze pamiętając o testowaniu planu wykonania za pomocą EXPLAIN, aby zapewnić skalowalność w środowiskach produkcyjnych zawierających miliony rekordów.

Machine Translated by Google

Moduł 9: Funkcje składowane i procedury

Widoki stanowią podstawowe narzędzie w ramach relacyjnych baz danych, ponieważ oferują elastyczny sposób prezentacji danych użytkownikom i aplikacjom bez konieczności fizycznego przechowywania dodatkowych informacji.
Zasadniczo widok działa jak predefiniowane zapytanie, które system traktuje tak, jakby było tabelą wirtualną.

Gdy użytkownik wysyła zapytanie do widoku, moduł bazy danych wykonuje zapytanie zapisane w tle i dostarcza wyniki w czasie rzeczywistym, zapewniając zawsze wyświetlanie najbardziej aktualnych danych z oryginalnych tabel. Ta funkcja wyraźnie odróżnia widoki od rzeczywistych tabel, ponieważ nie zajmują one oddzielnej przestrzeni dyskowej, z wyjątkiem szczególnych przypadków.

Utworzenie widoku wymaga użycia polecenia CREATE VIEW, a następnie wybrania nazwy widoku i definicji zapytania SELECT, które określa, jakie dane mają zostać wyświetlone. Zaleca się rozpoczęcie od prostych przykładów, aby zrozumieć, jak to działa.

Załóżmy, że mamy bazę danych do zarządzania sklepem internetowym z
Tabela o nazwie „Products" zawiera kolumny takie jak: ProductID, Name, Price, Stock, CategoryID i AddedDate. Jeśli raporty sprzedaży często muszą wyświetlać tylko nazwę, cenę i dostępny stan magazynowy produktów, bez ujawniania pozostałych informacji o produkcie, należy skorzystać z tej tabeli.
Informacje wewnętrzne, takie jak data dodania, można zdefiniować w następującym widoku:

```
UTWÓRZ WIDOK Zobacz ProduktyDostępne JAKO
WYBIERZ ID produktu, nazwę, cenę, stan magazynowy
OD produktów
GDZIE Zapas > 0;
```

Po wykonaniu tej instrukcji programiści lub analitycy mogą tworzyć zapytania do widoku tak, jak do zwykłej tabeli. Na przykład zapytanie takie jak „SELECT Nazwa, Cena FROM DostępnyWidokProduktów ORDER BY Cena DESC" pozwoliłoby na wyszukanie najdroższych dostępnych produktów bez konieczności każdorazowego pisania filtra zapasów. Takie podejście znacznie upraszcza codzienną pracę i zmniejsza liczbę błędów poprzez standaryzację dostępu do określonych zestawów danych.

Zalety wdrażania widoków są liczne i obejmują aspekty bezpieczeństwa, konserwacji i wydajności. Z perspektywy bezpieczeństwa widoki umożliwiają
Precyzyjnie kontroluj, jakie informacje widzą poszczególni użytkownicy. Zamiast przyznawać pełne uprawnienia do tabeli „Produkty", administrator może przyznać uprawnienia SELECT.

Machine Translated by Google

Ten widok wyświetla tylko wrażliwe kolumny, takie jak koszty dostawcy lub powiązane dane klienta, ukrywając je. Jest to szczególnie przydatne w środowiskach z przepisami o ochronie danych, ponieważ umożliwia tworzenie widoków filtrujących wiersze na podstawie ról użytkownika, na przykład wyświetlając tylko produkty z określonych kategorii dla określonych dostawców.

Jeśli chodzi o konserwację, widoki pełnią funkcję bardzo rozbudowanej warstwy abstrakcji. Jeśli w przyszłości tabela „Products" zmieni swoją strukturę – na przykład poprzez podział na kilka tabel w celu lepszej normalizacji – wystarczy zaktualizować definicję widoku, zachowując tę samą nazwę i te same widoczne kolumny. W ten sposób wszystkie aplikacje, raporty i procedury zależne od tego widoku będą nadal działać bez konieczności wprowadzania zmian w kodzie klienta.

Ta niezależność między strukturą fizyczną i logiką biznesową jest jedną z największych zalet w długoterminowych projektach, w których baza danych podlega ciągłej ewolucji.

Kolejnym istotnym aspektem jest wpływ na produktywność zespołu programistów. Złożone zapytania obejmujące wiele łączeń, obliczeń lub filtrów można hermetyzować.
w widoku, dzięki czemu kod aplikacji jest bardziej czytelny i łatwiejszy w utrzymaniu. Rozważmy bardziej zaawansowany scenariusz, w którym trzeba połączyć

Informacje o produktach z kategoriami i dostawcami. Można utworzyć następujący widok:

```
UTWÓRZ WIDOK ZobaczProduktySzczegółowe JAKO
WYBIERZ p.ProductoID, p.Nombre, p.Precio, c.NombreCategoria, pr.NombreProveedor, p.Stock
OD Produkty p
POŁĄCZENIE WEWNĘTRZNE Kategorie c NA p.CategoriaID = c.CategoriaID
INNER JOIN Dostawcy pr ON p.SupplierID = pr.SupplierID
GDZIE p.Active = 1;
```

Teraz każde zapytanie wymagające połączonych danych może po prostu odwołać się do VistaProductosDetallados, unikając konieczności stosowania skomplikowanych połączeń w różnych częściach systemu. Co więcej, jeśli później zostanie wprowadzona zmiana w sposobie obliczania kolumny pochodnej, modyfikacja zostanie wprowadzona w jednym miejscu.

Istnieją przypadki, w których widoki można aktualizować, co oznacza, że można bezpośrednio na nich wykonywać polecenia INSERT, UPDATE lub DELETE i odzwierciedlać te zmiany w tabelach bazowych.

Dzieje się tak głównie wtedy, gdy widok jest oparty na pojedynczej tabeli i nie zawiera złożonych agregacji ani łączeń. W pierwszym przykładzie widoku AvailableProductsView możliwe byłoby zaktualizowanie stanu magazynowego produktu za pomocą widoku: UPDATE AvailableProductsView SET Stock = Stock - 1
WHERE ProductID = 5;.

Machine Translated by Google

Jeśli jednak widok obejmuje wiele tabel, jak w szczegółowym przykładzie, zazwyczaj staje się on dostępny tylko do odczytu, chyba że zaimplementowano wyzwalacze lub użyto specjalnych opcji silnika bazy danych.

W scenariuszach wymagających wysokiej wydajności niektóre systemy, takie jak SQL Server, umożliwiają tworzenie widoków indeksowanych. Te zmaterializowane widoki fizycznie przechowują wyniki zapytania na dysku i są automatycznie aktualizowane po zmianie danych bazowych.

Chociaż pochłaniają więcej zasobów pamięci masowej i wymagają dodatkowej konserwacji, widoki indeksowane są niezwykle przydatne w magazynach danych lub podczas wykonywania dużych zapytań analitycznych z częstymi agregacjami. Utworzenie widoku indeksowanego polega najpierw na zdefiniowaniu widoku za pomocą wiązania schematu, a następnie utworzeniu na nim unikalnego indeksu klastrowego.

Pomimo wszystkich tych zalet, ważne jest, aby używać widoków rozsądnie. Źle zaprojektowany widok, który zawiera bardzo kosztowne zapytania, może obniżyć ogólną wydajność systemu, jeśli jest często odpytywany. Podobnie, nadużywanie widoków zagnieżdżonych (widoków, które...
konsultowanie innych poglądów) może utrudniać diagnozę problemów z wydajnością.

Dlatego zawsze warto przeanalizować plan wykonania zapytań bazowych i rozważyć alternatywy, takie jak procedury składowane, gdy potrzebna jest bardziej złożona logika. Ogólnie rzecz biorąc, widoki wyróżniają się możliwością łączenia prostoty, bezpieczeństwa i elastyczności w obsłudze danych.

Po ustanowieniu tej warstwy abstrakcji za pomocą widoków, kolejny poziom hermetyzacji logiki w bazie danych zapewniają funkcje zdefiniowane przez użytkownika, znane jako funkcje definiowane przez użytkownika (UDF). Funkcje te umożliwiają tworzenie niestandardowych obliczeń lub transformacji, które można wywołać bezpośrednio w dowolnym zapytaniu SELECT, podobnie jak wbudowane funkcje silnika, takie jak GETDATE czy LEN. W przeciwieństwie do widoków, które zwracają całe zestawy wierszy, funkcje definiowane przez użytkownika (UDF) są zaprojektowane tak, aby zwracać pojedynczą wartość lub całą tabelę, w zależności od ich zdefiniowanego typu. Dzięki temu idealnie nadają się do ponownego wykorzystywania złożonych reguł biznesowych bez konieczności powtarzania kodu w wielu miejscach.

Tworzenie funkcji skalarnej, najprostszego typu, odbywa się za pomocą instrukcji CREATE FUNCTION, określając parametry wejściowe w nawiasach, typ danych zwracanych oraz treść funkcji między BEGIN a END. Załóżmy, że w naszym sklepie internetowym często potrzebujemy obliczyć cenę z rabatem na podstawie zmiennej promocji. Możemy zdefiniować taką funkcję:

```
UTWÓRZ FUNKCJĘ CalcularPriceWithDiscount (@PriceBase DECIMAL(10,2),
@DiscountPercentage INT)
ZWRACA DECIMAL(10,2)
AS
ZACZYNAĆ
        ZWROT @BasePrice * (1 - @DiscountPercentage / 100,0);
KONIEC;
```

Machine Translated by Google

Używając go w zapytaniu, wystarczy napisać SELECT Nombre, dbo.CalcularPrecioConDescuento(Precio, 15) AS PrecioPromocion FROM Productos;. Silnik wykonuje funkcję wiersz po wierszu i zwraca obliczony wynik, zachowując przejrzystość zapytania i centralizując logikę rabatowania.

Możliwość ponownego wykorzystania jest kluczową zaletą, ponieważ jakakolwiek zmiana wzoru rabatowego, np. dodanie podatków, jest wykonywana tylko raz w ramach funkcji i jest automatycznie propagowana do wszystkich raportów i procedur, które ją wywołują.

Oprócz funkcji skalarnych istnieją funkcje tabelaryczne, które zwracają kompletny zestaw wyników i mogą być traktowane jak tabela w klauzuli FROM zapytania. Istnieją dwa główne warianty: funkcje wbudowane, które są zasadniczo opakowanym zapytaniem SELECT, oraz funkcje wieloinstrukcyjne, które pozwalają na bardziej rozbudowaną logikę proceduralną w ramach funkcji. Praktycznym przykładem wbudowanej funkcji tabelarycznej byłaby funkcja zwracająca wszystkie produkty w kategorii wraz z prognozowanymi stanami magazynowymi.

```
UTWÓRZ FUNKCJĘ GetProductsByCategory (@CategoriaID INT)
ZWRACA TABELA
AS
POWRÓT
(
    WYBIERZ Nazwa, Akcje, Cena
    OD produktów
    GDZIE CategoryID = @CategoriaID I Stock > 0
);
```

Aby z niego skorzystać, należy uruchomić polecenie `SELECT * FROM dbo.ObtenerProductosPorCategoria(3);`, a wynik zachowuje się dokładnie tak samo jak tabela tymczasowa. Jest to szczególnie przydatne. Przydatne, gdy trzeba uwzględnić złożone filtry powtarzające się w wielu raportach, ponieważ funkcja zachowuje ten sam interfejs, nawet jeśli wewnętrzna logika ulega zmianie.

W przypadku funkcji wieloinstrukcyjnych z wartościami tabelarycznymi można zadeklarować zmienną typu TABLE, wypełnić ją kilkoma poleceniami INSERT i zwrócić, umożliwiając w razie potrzeby wykonywanie pośrednich obliczeń lub pętli.

Funkcje definiowane przez użytkownika oferują znaczną przewagę pod względem modułowości i wydajności, ponieważ silnik bazy danych może optymalizować ich wykonywanie i buforować plany zapytań. Należy jednak pamiętać o ich ograniczeniach: funkcje skalarne nie mogą modyfikować danych w bazie danych ani wykonywać poleceń takich jak INSERT czy UPDATE, ponieważ są przeznaczone wyłącznie do obliczeń.

To wyraźnie odróżnia je od procedur składowanych, które umożliwiają pełne operacje manipulacji danymi. Ponadto, w środowiskach o dużej liczbie transakcji, zaleca się unikanie skorelowanych funkcji skalarnych w instrukcji SELECT, ponieważ mogą one spowalniać zapytania; w takich przypadkach lepiej jest przekształcić je w połączenia za pomocą funkcji tabelarycznych.

Machine Translated by Google

Z drugiej strony, wbudowane funkcje tabelaryczne zachowują się niemal jak widoki parametryczne i są rozwijane wewnętrznie przez optymalizator, co zwykle zapewnia lepszą wydajność niż funkcje wielozadaniowe.

W praktyce wielu programistów zaczyna od tworzenia funkcji skalarnych do prostych obliczeń, a następnie, w miarę rozwoju logiki, przechodzą na procedury składowane lub funkcje tabelaryczne, w zależności od przypadku użycia.

Taka elastyczność sprawia, że funkcje UDF stanowią doskonały pomost między widokami statycznymi i bardziej zaawansowaną logiką programistyczną w samym silniku SQL Server.

Gdy potrzeby wykraczają poza proste obliczenia lub zwrócenie zestawu danych i wymagają wykonania kompletnej sekwencji instrukcji, transakcji lub logiki warunkowej, która oddziałuje na wiele tabel, idealnym narzędziem stają się procedury składowane.

Procedura składowana to zestaw skompilowanych instrukcji T-SQL przechowywanych w bazie danych, które są wykonywane na żądanie za pomocą instrukcji EXEC lub EXECUTE. W przeciwieństwie do funkcji, procedury mogą modyfikować dane, zwracać wiele wartości wyjściowych i obsługiwać błędy w sposób strukturalny, co czyni je idealnymi do hermetyzacji całych procesów biznesowych, takich jak rejestrowanie sprzedaży, aktualizacja zapasów i generowanie dziennika audytu, w jednym wywołaniu.

Procedurę tworzy się za pomocą polecenia CREATE PROCEDURE, po którym następuje nazwa i lista parametrów w nawiasach. Każdy parametr może być parametrem wejściowym (domyślnie IN), wyjściowym (OUT) lub jednym i drugim (parametr INOUT jest niejawny w przypadku polecenia OUTPUT). Dla przykładu, załóżmy, że chcemy utworzyć procedurę, która aktualizuje stan magazynowy produktu po sprzedaży i zwraca pozostały stan magazynowy jako parametr wyjściowy. Kod wyglądałby mniej więcej tak:

```
UTWÓRZ PROCEDURĘ AktualizacjaSprzedażyZapasów
    @ProductID INT,
    @QuantitySold INT,
    @StockRemaining DECIMAL(10,2) WYJŚCIE
AS
ZACZYNAĆ
    ZACZNIJ SPRÓBOWAĆ
        ROZPOCZNIJ TRANSAKCJĘ;

        AKTUALIZACJA Produkty
        USTAW Zapasy = Zapasy - @QuantitySold
        GDZIE ProductID = @ProductID;

        WYBIERZ @StockRestante = Stock
        OD produktów
```

Machine Translated by Google

```
            GDZIE ProductID = @ProductID;

            ZATWIERDŹ TRANSAKCJĘ;
        KONIEC PRÓBY
        ROZPOCZNIJ ŁAPANIE
            WYCOFANIE TRANSAKCJI;
            RZUCIĆ;
        ZACZEP KOŃCOWY
KONIEC;
```

Aby wywołać go z innego zapytania lub aplikacji, wpisz:

```
DEKLARUJ @StockFinal DECIMAL(10,2);
EXEC UpdateSale @ProductID = 5, @QuantitySold = 3, @RemainingStock
= @StockFinal WYJŚCIE;
WYBIERZ @StockFinal JAKO StockAfterSale;
```

Parametr wyjściowy pozwala procedurze zwrócić obliczone informacje bez konieczności końcowego polecenia SELECT, co jest bardzo praktyczne, gdy aplikacja musi bezpośrednio przechwycić tę wartość. Wartości domyślne można zdefiniować dla parametrów wejściowych, takich jak @QuantitySold INT = 1, co zwiększa elastyczność procedury, umożliwiając wywołania z mniejszą liczbą argumentów przy użyciu wartości domyślnej.

Kolejną znaczącą zaletą procedur składowanych jest pamięć podręczna planu wykonania: po skompilowaniu plan jest zapisywany i ponownie wykorzystywany w kolejnych wywołaniach, co znacznie poprawia wydajność w porównaniu do wysyłania tej samej logiki SQL z aplikacji za każdym razem.

Ponadto zapewniają dodatkową warstwę bezpieczeństwa, ponieważ można udzielać uprawnień Wykonanie procedury bez udzielania bezpośredniego dostępu do tabel bazowych. Użytkownik mógłby wykonać procedurę UpdateStockSale, ale nigdy nie mógłby wykonać bezpośredniej aktualizacji tabeli Products, co jest zgodne z zasadą najmniejszych uprawnień.

Procedury obsługują również parametry z wartościami tabelarycznymi, wprowadzone we współczesnych wersjach SQL Server, które umożliwiają przekazywanie całych zestawów danych w jednym wywołaniu. Na przykład, można utworzyć typ tabeli dla wierszy zamówienia i przekazać go do procedury, która wstawia wiele produktów do sprzedaży jednocześnie, zmniejszając liczbę cykli przesyłania danych do bazy danych.

Możliwość ta sprawia, że procedury są preferowaną opcją w przypadku przetwarzania wsadowego lub integracji z aplikacjami zewnętrznymi.

Wyzwalacze, zwane również wyzwalaczami, stanowią kolejną warstwę automatyzacji w bazie danych. Wyzwalacz to specjalna procedura, która

Machine Translated by Google

Działa automatycznie w odpowiedzi na określone zdarzenie, bez konieczności
Nikt nie powinien tego wywoływać wprost. Najczęstszymi zdarzeniami są operacje DML: INSERT, UPDATE lub DELETE w tabeli lub widoku. Można je skonfigurować tak, aby wyzwalały PRZED (w niektórych silnikach) lub PO zakończeniu operacji, a nawet INSTEAD OF, aby całkowicie zastąpić domyślne zachowanie.

Tworzenie odbywa się za pomocą polecenia CREATE TRIGGER, które określa nazwę tabeli, na której znajduje się który działa i czas wykonania. W wyzwalaczu znajdują się dwie specjalne tabele wirtualne o nazwach INSERTED i DELETED, które zawierają wiersze, których to dotyczy.
Kontynuując przykład naszego sklepu, moglibyśmy utworzyć wyzwalacz, który automatycznie zapisuje w tabeli audytu każdą aktualizację ceny produktu:

```
UTWÓRZ WYZWALACZ trg_AuditoryProductPrice
Produkty ON
PO AKTUALIZACJI
AS
ZACZYNAĆ
    JEŚLI AKTUALIZACJA (cena)
    ZACZYNAĆ
        WSTAW DO AuditiaPrecios (ProductoID, PrecioAnterior, PrecioNuevo,
Zmiana daty)
        SELECT d.ProductoID, d.Precio, i.Precio, GETDATE()
        Z USUNIĘTEGO d
        WSTAWIONE POŁĄCZENIE WEWNĘTRZNE i NA d.ProductID = i.ProductID
        GDZIE d.Cena <> i.Cena;
    KONIEC
KONIEC;
```

Za każdym razem, gdy ktoś wykona operację UPDATE w kolumnie „Cena”, wyzwalacz uruchomi się automatycznie, porówna stare i nowe wartości oraz zarejestruje zmianę. Pozwala to na wdrażanie reguł biznesowych lub audytów bez konieczności modyfikowania kodu aplikacji, które wykonują aktualizacje.

Innym powszechnym zastosowaniem jest walidacja danych: wyzwalacz AFTER INSERT może sprawdzać, czy kurs akcji nie stanie się ujemny, a jeśli tak się stanie, zgłosić błąd RAISERROR i anulować transakcję.

Wyzwalacze mogą być zagnieżdżane (jeden wyzwalacz uruchamia inny) i mogą być również dezaktywowane. Tymczasowo użyj opcji WYŁĄCZ WYZWALACZ podczas ładowania danych zbiorczych, aby poprawić wydajność. Zaleca się jednak oszczędne korzystanie z nich, ponieważ uruchamianie ich przy każdej operacji może mieć wpływ na ogólną wydajność, jeśli zawierają...

Machine Translated by Google

Ciężka logika. W praktyce wyzwalacze idealnie sprawdzają się w utrzymywaniu złożonej integralności referencyjnej, synchronizowaniu tabel podsumowujących lub implementacji rejestrowania zmian bez dotknij kodu aplikacji.

Aby zbudować całą tę logikę w procedurach, funkcjach i wyzwalaczach, niezbędne jest opanowanie obsługi zmiennych i struktur sterujących. Zmienne deklaruje się za pomocą polecenia DECLARE, po którym następuje nazwa i typ danych, a wartości przypisuje się za pomocą polecenia SET lub SELECT. Na przykład, w ramach procedury można zadeklarować polecenie DECLARE @Counter INT = 0;, a następnie modyfikować jego wartość w razie potrzeby. Pozwala to na przechowywanie wyników pośrednich, liczników lub wartości konfiguracyjnych, które są ponownie wykorzystywane w całym kodzie.

Najczęściej używanymi strukturami sterującymi są IF...ELSE dla decyzji warunkowych, WHILE dla pętli oraz CASE dla wielokrotnych ocen. Załóżmy, że w ramach procedury aktualizacji zapasów chcemy zastosować dodatkową logikę: jeśli zapasy spadną poniżej 10 jednostek, wyślij alert lub wstaw rekord do tabeli produktów o niskim stanie magazynowym. Fragment kodu wyglądałby następująco:

```
JEŻELI @StockRestante < 10
ZACZYNAĆ
    WSTAW DO Alertów niskiego stanu magazynowego (ID produktu, Aktualny stan magazynowy, Data alertu)
    WARTOŚCI (@ProductID, @RemainingStock, GETDATE());
KONIEC
W PRZECIWNYM RAZIE
ZACZYNAĆ
    DRUKUJ 'Wystarczająca ilość zapasów';
KONIEC
```

Blok BEGIN...END pozwala grupować wiele instrukcji. W przypadku procesów powtarzalnych pętla WHILE jest bardzo praktyczna. Wyobraź sobie procedurę przetwarzającą partię. produktów i aktualizuj ceny jeden po drugim, aż do zakończenia:

SQL

```
DEKLARUJ @ProductoID INT = 1;
GDY @ProductoID <= 100
ZACZYNAĆ
    AKTUALIZACJA Produkty USTAW Cena = Cena * 1,05 GDZIE ProductID = @ProductID;
    USTAW @ProductID = @ProductID + 1;
KONIEC
```

Chociaż operacje oparte na zbiorach są w praktyce bardziej wydajne, pętle WHILE są przydatne w scenariuszach przetwarzania sekwencyjnego lub migracji. Struktury CASE z kolei mogą być używane zarówno w zapytaniach SELECT, jak i w blokach sterujących.
W ramach procedury można zapisać:

SQL

Machine Translated by Google

```
DEKLARUJ @Poziom CHAR(1) =
        SPRAWA
                GDY @StockRemaining > 50 WTEDY 'A'
                GDY @StockRestante MIĘDZY 10 A 50 , WTEDY 'B'
                W PRZECIWNYM RAZIE 'C'
        KONIEC;
```

Taka elastyczność pozwala na tworzenie niezwykle solidnych procedur, które reagują na różne scenariusze bez konieczności stosowania wielu oddzielnych procedur. Wszystkie te konstrukcje są naturalnie łączone w ramach funkcji, procedur i wyzwalaczy. umożliwiając wdrażanie kompletnych rozwiązań biznesowych bezpośrednio do bazy danych w sposób wydajny i łatwy w utrzymaniu.